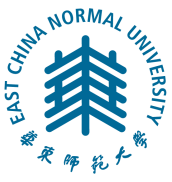
2020届研究生硕士学位论文

分 类 号：_____ 学校代码：10269

密 级：_____ 学 号：51170601048



East China Normal University

硕士学位论文

MASTER'S DISSERTATION

论文题目：基于深度强化学习的二维矩形条装箱问题求解算法

院 系：数学科学学院

专 业：应用数学

研 究 方 向：图论及其应用

指 导 老 师：吕长虹 教授

学位申请人：来蕾

2020 年 05 月

中国知网 https://www.cnki.net

Dissertation for master degree in 2020 University Code: 10269

Student ID: 51170601048

East China Normal University

Title: Solving the 2D Strip Packing Problem with Deep Reinforcement Learning

Department: School of Mathematical Sciences

Major: Applied Mathematics

Research Direction: Graph Theory and Its Application

Supervisor: Lu Changhong (Professor)

Candidate: Lai Lei

05/2020

中国知网 https://www.cnki.net

中国知网 https://www.cnki.net

来蕾 硕士学位论文答辩委员会成员名单

姓 名	职 称	单 位	备 注
张晓东	教授	上海交通大学数学科学学院	主席
任韩	教授	华东师范大学数学科学学院	
詹兴致	教授	华东师范大学数学科学学院	

中国知网 https://www.cnki.net

摘　要

二维矩形条装箱问题 (Two-Dimensional Strip Packing Problem, 2D-SPP)：给定矩形条的宽度 W 固定，高度 H 不固定，以及一组小矩形 $\{(w_1, h_1), (w_2, h_2), ..., (w_n, h_n)\}$，其中小矩形宽度为 w_i，高度为 h_i，$1 \leq i \leq n$，$0 \leq h_i$，$0 < w_i \leq W$，目标将所有小矩形放置入矩形条中，且小矩形不能旋转，小矩形之间不能重叠，并最小化矩形条的高度 H。常用于求解 2D-SPP 的算法主要有精确算法，启发式算法。本文提出一种基于深度强化学习的 2D-SPP 求解算法，在智能体与环境不断交互的过程中学习策略以达到期望累计奖励最大化，即最小化矩形条的高度 H。

深度强化学习是深度学习与强化学习相结合的产物，本文选择基于策略的强化学习方法作为本文的强化学习算法，深度神经网络模型 Pointer Networks 作为强化学习算法的策略函数，随机性策略作为策略函数的输出，累计奖励的期望值作为目标函数。Pointer Networks 是 Seq2Seq 结合改进的注意力机制后得到的模型，可以有效地处理“输出严重依赖输入”类型的问题，非常适合作为本文 2D-SPP 求解算法的策略函数。在训练阶段，深度强化学习通过策略梯度算法计算目标函数的梯度值，并选择 Adam 优化算法更新 Pointer Networks 的模型参数。在测试阶段，策略函数 Pointer Networks 通过 Beam Search 算法搜索装箱策略，策略函数返回的装箱策略进一步结合构造类启发式 Bottom-Left-Fill 算法对小矩形进行装箱。

本文通过大量实验对基于深度强化学习的 2D-SPP 求解算法进行性能评估和测试，通过训练过程中累计奖励的期望值随训练步数的变化趋势分析算法的稳定性，通过测试过程中 Pointer Networks 针对测试样本返回的装箱策略分析算法的可行性，通过策略函数决策过程中输出概率分布的可视化结果分析算法的合理性。本文最后在 2D-SPP 的经典数据集上对本文算法效果进行验证，并与启发式搜索算法进行比较，实验表明基于深度强化学习的 2D-SPP 求解算法在实验结果普遍优于启发式搜索算法的同时，在求解大规模问题时，本文算法线上计算复杂度低，运行时间短，线上运行速度显著优于启发式搜索算法。本文通过实验展示了深度强化学习算法用于求解 2D-SPP 的可行性与优势，为今后求解装箱问题提供了一种全新的解决思路。

关键词: 二维矩形条装箱问题, Pointer Networks, 深度强化学习, 策略梯度算法

中国知网　https://www.cnki.net

ABSTRACT

Two-Dimensional Strip Packing Problem (2D-SPP): given a strip with fixed width W and variable height H, and a set of rectangles$\{(w_1, h_1), (w_2, h_2), ..., (w_n, h_n)\}$, each rectangles with width w_i and height h_i, $1 \leq i \leq n$， $0 \leq h_i$， $0 < w_i \leq W$, 2D-SPP is to pack all rectangles into the strip without rotating and overlapping and the goal is to minimize the height of used strip. Exact methods and Heuristic algorithms are widely applied to solve 2D-SPP. In this paper, we propose a deep reinforcement learning algorithm to solve 2D-SPP. Deep reinforcement learning is an aspect of machine learning where an agent learns to behave in an environment, by performing certain actions and observing the rewards which it gets from those actions.

Deep reinforcement learning is the product of a combination of deep learning and reinforcement learning. In this paper, we choose policy based reinforcement learning method as our algorithm, Pointer Networks as the policy function, stochastic policy as the output of our policy function and expected cumulative reward as our objective function. Pointer Networks is the model combines Seq2Seq with improved attention mechanism which works well on solving "output depends on the length of the input" problems, like 2D-SPP. During training, we apply Policy Gradient algorithm to calculate the gradient of objective function and Adam algorithm to update the parameters in Pointer Networks. During testing, Pointer Networks applies Beam Search algorithm to search the packing strategy. Constructive heuristic algorithm Bottom-Left-Fill packs all rectangles into the strip under the returned packing strategy.

In this paper, we analyze the stability, feasibility and rationality of our algorithm through experiments. We also test the performance of our algorithm on classic 2D-SPP data set C and compare with other heuristic search algorithms. Experiments show that during solving large-scale problem, our algorithm obtains better or equivalent results with much lower time complexity and faster running speed than heuristic search algorithms. Our experiments show the advantage of deep reinforcement learning algorithm on solving 2D-SPP, provide a brand new solution for solving bin packing problems.

Keywords: 2D Strip Packing Problem, Pointer Networks, Deep Reinforcement Learning, Policy Gradient

中国知网 https://www.cnki.net

目 录

摘　要 i

Abstract iii

第一章　绪论 1

1.1　课题背景 1
1.2　研究意义 2
1.3　研究现状 3
1.4　主要工作和本文结构 13
1.5　本章小结 15

第二章　深度神经网络模型 17

2.1　循环神经网络 17
2.2　Sequence to Sequence 模型 23
2.3　Pointer Networks 模型 25
2.4　Beam Search 算法 27
2.5　本章小结 29

第三章　基于深度强化学习的 2D-SPP 算法 31

3.1　强化学习 31
3.2　Policy Based 深度强化学习算法 33
3.3　算法优化 36
3.4　Bottom-Left-Fill 算法 40
3.5　本章小结 42

第四章　实验设计及结果分析 43

4.1　实验设计 43
4.2　实验结果分析 44
4.3　本章小结 50

总结与展望 53

中国知网 https://www.cnki.net

参考文献 55

附录 59

致谢 61

中国知网 https://www.cnki.net

第一章　绪论

1.1　课题背景

装箱问题是经典的离散组合最优化问题。装箱问题最早可以追溯到 1831 年高斯 (Gauss) 对布局问题的研究，到现在已有百余年的历史。从 20 世纪 70 年代初开始，装箱问题引起了学者广泛的探讨和研究。同时，装箱问题被广泛应用于运输业、制造业、计算机行业等，装箱问题的研究对促进工业发展起到不可或缺的作用。

在运输行业中，如集装箱货物的装载便是三维装箱问题 (Three-Dimensional Bin Packing Problem, 3D-BPP)。在此问题中，需要将若干个不同大小的长方体货物装入集装箱中，货物之间不能重叠且不能倾斜，货物的尺寸和成本已知，优化目标为最小化集装箱的使用数量，即最小化总成本。近年来，随着我国经济的高速发展，运输行业的实际业务场景在不断扩宽，一些新型的三维装箱问题正在不断提出并深入研究。例如在跨境电商业务中，大部分是使用柔性的塑料材料来包装货物，而不是使用固定尺寸的集装箱进行装载，且由于装箱的成本大部分都由装箱材料成本构成，而装箱材料成本又主要取决于材料的表面积，因此与传统的三维装箱问题不同，该场景下的优化目标为确定一个能够容纳所有物品的箱子，并最小化此箱子的表面积。中国是世界第二大经济体，港口的集装箱吞吐量，国际贸易的进出口量，都处于世界领先地位。因此，装箱问题的研究对中国运输行业的发展有至关重要的意义。

在制造行业中，二维装箱问题 (Two-Dimensional Bin Packing Problem, 2D-BPP) 有着广泛的应用。如木材或玻璃加工厂，给定一块较大的矩形木板（玻璃），厂商通常根据客户的需求，从这块较大的矩形木板（玻璃）上裁剪出给定的大小不一的小矩形木板（玻璃），求一种剪法使得剪出的废料的面积总和最小。还有在服装行业中，通过合理裁剪，在生产更多成衣的同时减少布料的浪费。这些都是典型的二维矩形条装箱问题 (Two-Dimensional Strip Packing Problem, 2D-SPP)。制造业中的诸多场景和装箱问题息息相关，通过对装箱问题的研究，可以为制造业节省大量资源。

在计算机行业中，如计算机的存储问题，假设要把大小不同的共 10MB 的

中国知网　https://www.cnki.net

文件拷贝到磁盘中去，而每张磁盘的容量为 1.44MB，已知每个文件的字节数不超过 1.44MB，而且一个文件不能分成几部分存储，如何用最少的磁盘张数完成。这便是典型的一维装箱问题 (One-Dimensional Bin Packing Problem, 1D-BPP)。除此之外，装箱问题在云计算中也有重要应用。若将计算机理解为一个拥有硬盘容量，CPU 处理能力，内存容量等属性的箱子，将客户对存储容量，计算能力等需求理解成需要装箱的货物，那么云计算便可以理解为一种装箱问题。装箱问题的学习研究对计算机行业诸多领域也有着重要影响。

因此，装箱问题有着重要的理论研究意义和实际应用价值。

1.2 研究意义

本文主要研究的是通过深度强化学习求解二维矩形条装箱问题，即矩形条的宽度固定，高度不固定，给定一组小矩形，目标是将所有小矩形装下，小矩形不能旋转，小矩形之间不能重叠，并最小化所使用的矩形条的高度，如图3.5所示。

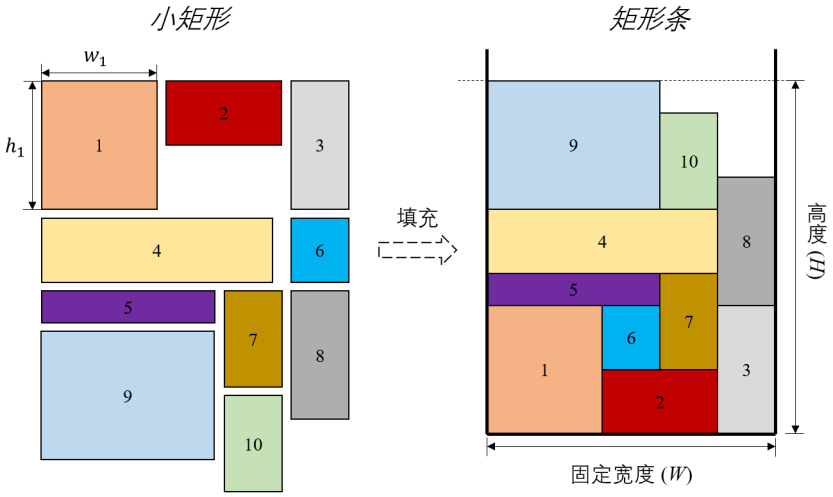


图 1.1 二维矩形条装箱问题

2D-SPP 已经被证明为 NP-Hard 问题 [19]。目前常用于求解 2D-SPP 的算法主要有精确算法，启发式算法和元启发式算法。随着人工智能的兴起，学习的方法在诸多领域取得巨大的成功，也给求解组合优化问题带来了新的启发。近年来出现一些运用深度学习和强化学习求解组合优化问题的工作。虽然这些方法还处于探索阶段，但是已经取得了一定的成果，这说明了由数据驱动的学习类算法

中国知网 https://www.cnki.net

的可行性，同时展示了基于线下训练、线上运行的学习类算法在求解 2D-SPP 上的发展潜力。

1.3 研究现状

目前常用于求解 2D-SPP 的算法主要有精确算法，启发式算法，其中启发式算法可分为构造类启发式算法和搜索类启发式算法。随着人工智能的兴起，学习类算法也正被逐渐应用于求解装箱问题。

1.3.1 精确算法

精确算法通常使用数学建模的方法建立数学模型（包括目标函数，决策变量以及约束条件），通过优化算法（单纯形法，分支定界，分支切割，割平面方法等）求解该数学模型并得出问题的最优解。解的好坏由模型建立的是否合理，优化算法的设计等因素决定。

Beasley (1985)[5] 提出了树搜索算法求解无一刀切的 2D-SPP，作者提出该问题的 0-1 整数规划的 Lagrangian 松弛，并将其用作树搜索过程的约束。Christofides (1995)[15] 提出了一种求解带一刀切的 2D-SPP，通过该问题的动态规划的状态空间进行松弛来确定搜索边界。Martello (2003)[32] 通过松弛求出搜索下界，并使用该下界通过分支定界法求解 2D-SPP。Cui (2008)[17] 针对带一刀切的 2D-SPP 提出了基于分支定界的递归结构的启发式递归算法，使用上下界的信息进行剪枝处理。

使用精确方法虽然可以求得最优解，可以从理论上证明求得的解是最优的，但随着问题规模的扩大（可能呈指数级或者阶乘级的增长），对于中等规模或者大规模的问题，在有限的时间内不可能求得最优解。这就需要在求解精度和运算时间之间有一个折衷和权衡。对于大规模的问题，我们不需要求得最优解，只需在短时间内求得次优解或者满意解，加之计算机性能的提升，便出现了启发式算法。

1.3.2 构造类启发式算法

启发式算法是以问题为导向程序，根据问题的特殊结构或者性质来改进解。一般情况下，启发式算法比精确算法更容易实现。

中国知网 https://www.cnki.net

构造类启发式算法从空解开始，并不断往当前解中添加元素进行扩展，直到获得完整解。在 2D-SPP 的情景中，就是小矩形依次放置到矩形条的可放置空间中。早先的一些构造类启发式算法提出一些计算复杂度低的放置策略，这些放置策略忽视了先前放置过程中产生的一些小的空白区域，这也是后续提出的构造类启发式算法不断改进的方向。其中，根据不同启发式算法中将小矩形放置入矩形条中的放置策略不同，可以将其分为基于定位、基于适合度、基于分层和基于轮廓四类启发式算法。

1.3.2.1 基于定位的启发式算法

基于定位的启发式算法是最早出现的用于求解 2D-SPP 的启发式算法。其中最早的同时也是最著名的就是由 Baker 等人 (1980)[4] 提出的 Bottom-Left (BL) 算法。Bottom-Left 算法的策略就是尽可能地将小矩形放置到最低最左的位置，该算法首先将小矩形放置到矩形条的右上方，并使小矩形不断交替地向下移动和向左移动，直至小矩形在当前所处位置不能继续移动，如图1.2所示。Bottom-Left 算法的复杂度是 $\mathcal{O}(n^2)$，其中 n 指所需放置的小矩形的个数。Bottom-Left 算法的优点是其向下向左的策略使得算法的执行效率很高，缺点则是该策略无法将小矩形填补到先前放置小矩形过程所产生的空白区域中, 导致可利用空间的浪费。Liu 和 Teng (1999)[30] 提出了 Improved Bottom-Left (iBL) 算法，与 Bottom-Left 算法的思路基本一致，但是向下移动操作拥有更高的优先级，即小矩形一直向下移动到不能继续时，则向左移动，但只向左移动一定距离使小矩形又允许继续向下移动，如图1.2所示。Improved Bottom-Left 算法提出了一种新的搜索策略，但同样无法对已产生的空白区域进行填补。

针对 Bottom-Left 算法的缺陷，Chazelle (1983)[13] 提出了 Bottom-Left-Fill (BLF) 算法，Bottom-Left-Fill 算法的放置策略在向下向左搜索合适位置的过程中会将先前放置小矩形过程所产生的空白区域也考虑在内，如图1.3所示。由于 Bottom-Left-Fill 算法中的位置搜索更加复杂，所以 Bottom-Left-Fill 算法的复杂度是 $\mathcal{O}(n^3)$，其中 n 指所需放置的小矩形的个数。由于 Bottom-Left-Fill 算法解决了空白区域无法填补的问题，因此一组小矩形在相同的放置顺序下通过这两种算法进行放置，Bottom-Left-Fill 算法的效果会等于或好于 Bottom-Left 算法。

以上的几种算法都聚焦在如何为当前的小矩形搜索最佳位置，因此算法效果很大程度上依赖于这组小矩形的先后放置顺序，同样的算法应用在不同的先后放

中国知网 https://www.cnki.net

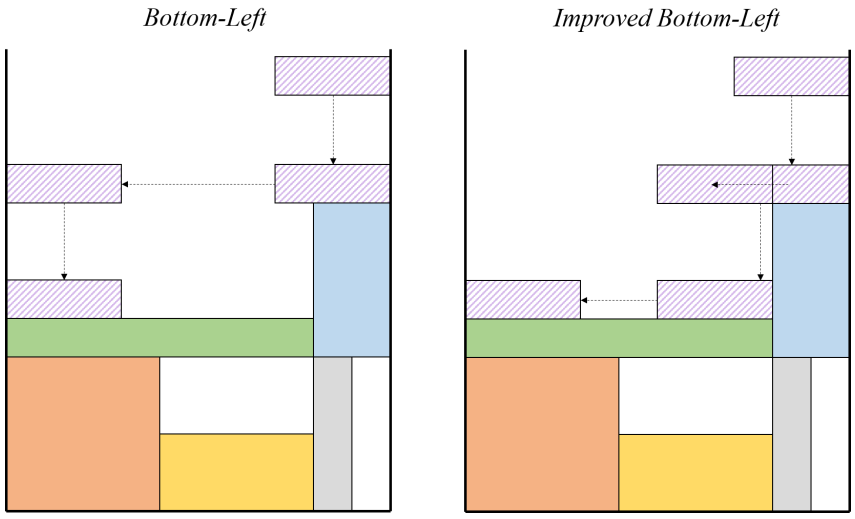


图 1.2 Bottom-Left 算法和 Improved Bottom-Left 算法

置顺序上将会有完全不一样的效果。为了降低算法对放置顺序的依赖性，Zhang 等人 (2016)[50] 在迭代算法“Priority Heuristic (PH)”[49] 的工作上进一步修改，动态赋予每个小矩形优先级，迭代算法会对矩形条依次切割成子矩形，并通过小矩形的先后放置顺序结合小矩形的优先级选择小矩形进行放置，该算法有效地降低了算法效果对放置顺序的依赖性。

1.3.2.2 基于适合度的启发式算法

基于适合度的启发式算法则将问题聚焦在矩形条的剩余可放置空间上。Burke 等人 (2004)[12] 提出了“Best-Fit (BF)”算法，通过剩余可放置空间来判断并选择合适的小矩形进行放置。“Best-Fit”算法通过引入 *skyline* 结构实现，*skyline* 由一系列满足以下条件的线段构成：(1) 每条线段都平行于 x 轴，(2) 相邻两条线段的 y 坐标值不同且有一个相同的 x 坐标值（即一条线段右端点的 x 坐标值等于另一条线段左端点的 x 坐标值），(3) 从无穷高处俯视，所有线段都没有被已放置的小矩形遮挡住，(4) 每一条线段的 y 坐标值等于某个已放置的小矩形顶边的 y 坐标值或等于 0，即矩形条底边的 y 坐标值。“Best-Fit”算法不断执行以下两个操作直到所有小矩形都放置入矩形条中：(1) 从当前 *skyline* 结构中找出可放置小矩形的最低的线段作为可放置空间，(2) 寻找可完美匹配可放置空间的小矩形，若不存在完美匹配的小矩形，则选择可放置入该空间的所有小矩形中最大的一个进行放置并提供三种放置规则：靠近空间最左侧进行放置、靠近空间最高侧矩形进行放置、靠近空间最低侧矩形进行放置，如图1.4所示。该算法的空间复杂

中国知网 https://www.cnki.net

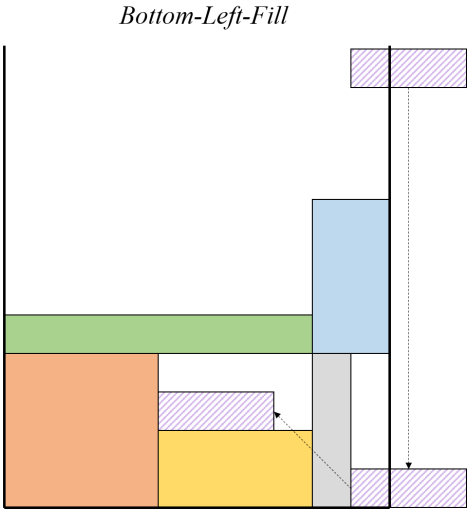


图 1.3 Bottom-Left-Fill 算法

度为 $\mathcal{O}(n+W)$，时间复杂度为 $\mathcal{O}(n^2+nW)$，其中 n 是小矩形的总数，W 是矩形条的宽度。

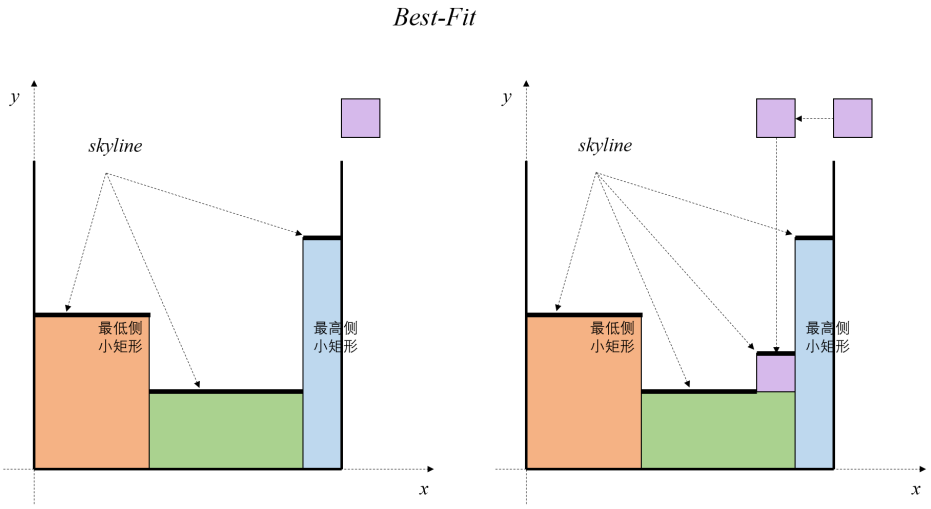


图 1.4 Best-Fit 算法

Imahori 和 Yagiura (2010)[25] 提出了一种“Best-Fit”算法更高效的实现方法。该实现方法将 *skyline* 的线段以其 y 坐标值为键存储到堆中，同时将这些线段按照其 x 坐标值排序存储到双向链表中，并通过二叉搜索树搜索小矩形，作者给出了空间复杂度为 $\mathcal{O}(n)$，时间复杂度为 $\mathcal{O}(nlogn)$，并证明了该时空复杂度是最优的。Verstichel 等人 (2013)[42] 采用了与 [25] 相同的数据结构，在“Best-Fit”算法的基础上，添加了三种新的放置标准，提出了“Three-way Best-Fit (T-w BF)”算法。

中国知网 https://www.cnki.net

Asık 和 Özcan (2009)[42] 在“Best-Fit”算法的工作上做了另一种改进，提出了“Bidirectional Best-Fit (BBF)”算法。“Bidirectional Best-Fit”算法在考虑已放置矩形所围出来的可放置空间的同时，还将先前放置矩形的上方空间考虑在内。Özcan 等人 (2013)[36] 在 [42] 的基础上提出了“Modified Bidirectional Best-Fit (BBFM)”算法，改进的算法通过几何相似度将小矩形分成不同群组，并将同一群组的小矩形组合并放置入矩形条内。

基于适合度的启发式算法中常见的缺陷是在放置小矩形的过程中，会不断产生一些很小的可放置空间（不存在小矩形可以放置入内，为无用空间），而这些空间的存在大大降低了算法的匹配效率。针对这个缺陷，Leung 和 Zhang (2011)[28] 提出了“Fast layer-based Heuristic (FH)”算法，该算法通过尽可能地设计一条“平坦”的 *skyline* 来消除这些不适用的空间，提高执行效率。算法根据可放置空间对等待放置小矩形进行适合度打分，最高适合度为 4（即小矩形完美匹配可放置空间），最低适合度为 0，适合度通过消除或最少增加可放置区域的边数计算所得，如图1.5所示。算法选择适合度最高的小矩形进行放置，放置的小矩形适合度越高，*skyline* 越“平坦”。后续有学者对适合度的评分机制进行了进一步的研究 [29, 45, 48]。

1.3.2.3 基于分层的启发式算法

与先前的算法不同，基于分层的启发式算法的思路是将矩形条平行于底边一刀切进行分层，并在每一层上放置小矩形。每一层的高度取由放置在该层的最高的小矩形的高度决定。这些基于分层的启发式算法在实际生活中高效地解决了受分层约束的装箱问题，如规划超市货架上商品的摆放问题。

Coman Jr. 等人 (1980)[16] 提出了“Next-Fit Decreasing Height (NFDH)”算法，算法对所有小矩形按照高度从高到低排序，在当前层依次靠左放置排序后的小矩形直至没有可放置空间，当前层没有可放置空间时则对矩形条添加新的分层。NFDH 算法的缺点是空间上的浪费，之前层的剩余空间没有利用起来放置后续一些小矩形。因此，Coman Jr. 等人 (1980)[16] 针对该缺陷又提出了“First-Fit Decreasing Height (FFDH)”算法，只要还有可放置空间，后续的小矩形也可以放置到之前的层中，只有当小矩形无法放置在之前所有分层时算法才添加一个新的分层，如图1.6所示。Berkey 和 Wang (1987)[7] 后续提出了“Best-Fit Decreasing Height (BFDH)”算法，当当前小矩形适合放置到之前的若干个层时，算法选择

中国知网 https://www.cnki.net

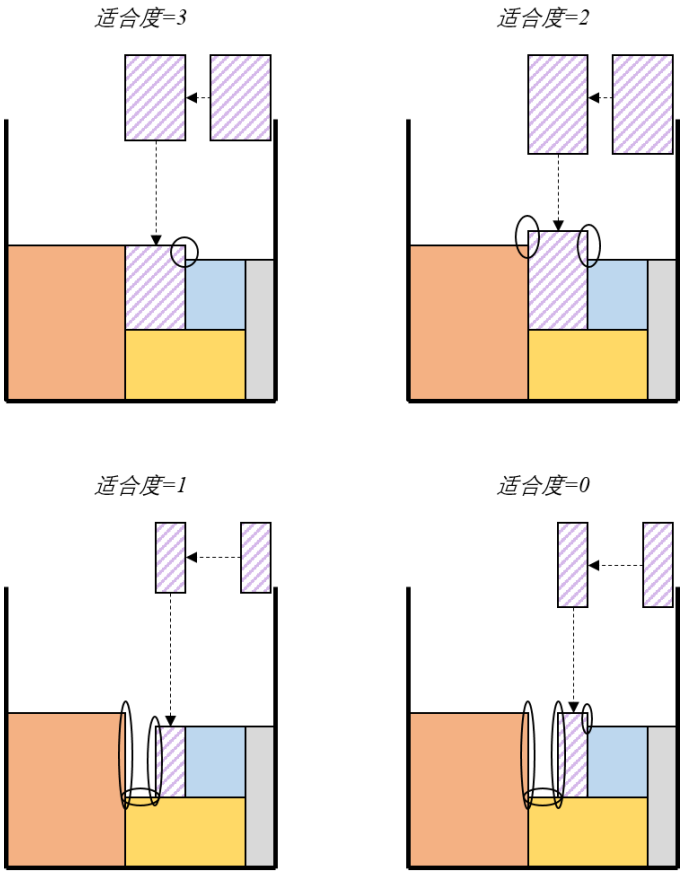


图 1.5 Fast layer-based 启发式算法

放置后未使用水平空间最小的层进行放置，而不是直接选择最低的层进行放置。

上述三种算法首先将小矩形按照高度从高到低排序，因此序列中相邻的两个小矩形有极大概率放置在同一层中，由于当前层的高度由该层中最高的矩形的高度决定，如果这两个相邻小矩形的高度差异很大，则会导致算法效果很差。为解决该问题，Ntene (2007)[34] 提出了“Size Alternating Stack (SAS)”算法。该算法将所有小矩形分成两个序列，第一个序列包含所有高度大于宽度的小矩形，第二个序列包含所有宽度大于等于高度的小矩形，第一个序列的小矩形按照高度降序排列，而第二个序列的小矩形按照宽度降序排列。每一层首先选择两个序列中最高的一个小矩形，根据该小矩形的高度设定该层的高度，并允许接下来放置的小矩形摆在之前小矩形的上方，只要不超过该层设定的高度。Ortmann 等人 (2010)[35] 在该工作的基础上，结合了 Lodi 等人 (1999)[31] 讨论的 floor ceiling 算法，提出了“Stack Ceiling with Re-sorting algorithm (SCR)”算法。

基于分层的启发式算法的研究至今依旧火热，如 Buchwald 和 Scheithauer

中国知网 https://www.cnki.net

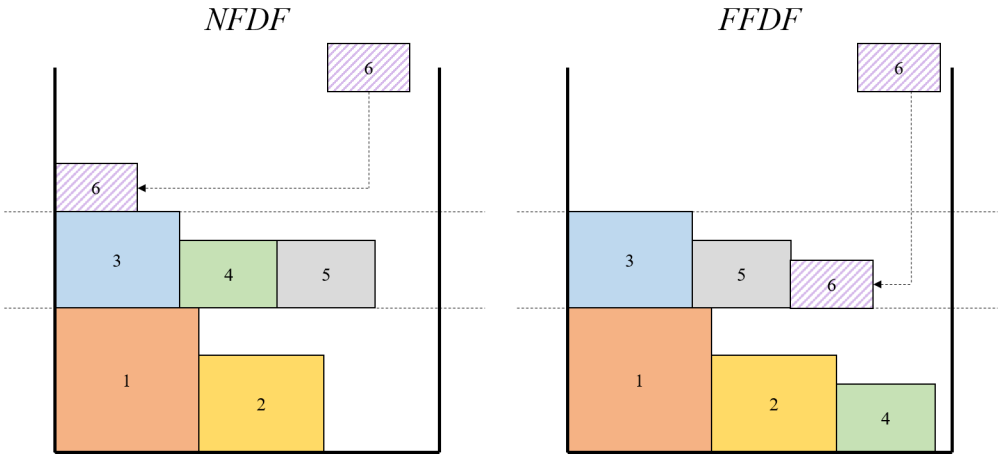


图 1.6 Next-Fit Decreasing Height 算法和 First-Fit Decreasing Height 算法

(2016)[9] 提出了“modified First-Fit Decreasing Height (FFDH*)”算法，作者在 FFDH 给出的结果基础上再对层进行旋转放置并给出了算法的理论结果。

1.3.2.4 基于轮廓的启发式算法

轮廓的概念最早由 Scheithauer (1997)[39] 提出，作者通过轮廓对已放置矩形进行描述，轮廓从矩形条左边开始至矩形条的右边或底边结束，由已放置小矩形的边或边的延申构成，而基于轮廓的放置策略使轮廓呈现阶梯状，如图1.7所示。Scheithauer 提出轮廓的概念并构造精确算法求解装箱问题，Pisinger (2007)[37] 提出了基于轮廓的启发式算法，作者通过序列对来描述已放置的小矩形，通过当前轮廓和拐角点来决定下一个小矩形的放置位置。

Wei 等人 (2011)[44] 基于轮廓的概念提出了“Iterative Doubling Binary Search (IDBS)”算法，不同于之前的放置策略，该论文提出放置策略使得轮廓呈 *skyline* 形状，向下向左放置的小矩形放置在 *skyline* 中线段的左端点，向下向右放置的小矩形放置在线段的右端点，与基于适合度的启发式算法相似，作者也对等待放置的小矩形基于浪费空间的大小进行了评估。

Zhang 等人 (2013)[51] 针对 Scheithauer 提出的轮廓进行了改进，并提出了“Binary Search Heuristic Algorithm (BSHA)”算法，算法根据所有待放置的小矩形形状评估，抛弃掉矩形条内的不可用空间，通过试验性放置小矩形计算放置后的浪费产生，并选择合适的小矩形进行放置。

中国知网 https://www.cnki.net

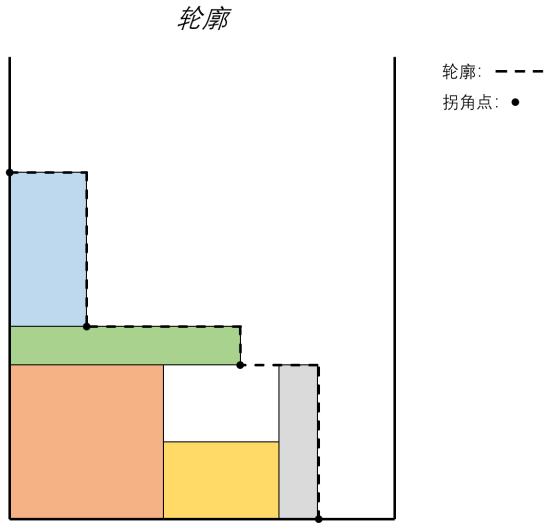


图 1.7 通过轮廓描述已放置矩形

1.3.3 搜索类启发式算法

搜索类启发式算法不同于构造类算法，由一个初始可行解开始（随机生成或由构造类算法产生），然后通过局部搜索迭代改进当前解。根据不同搜索类启发式算法中的搜索策略不同，可以将其分为基于序列搜索、基于布局搜索两类启发式算法。

1.3.3.1 基于序列搜索的启发式算法

基于序列搜索的求解 2D-SPP 的启发式算法通过交换和插入操作对序列进行修改，交换操作选取序列中的两个小矩形并交换它们在序列中的位置，插入操作提取序列中的一个小矩形并插入到序列的其它位置，并通过构造类算法依次将小矩形按照更新后的序列进行放置，如图1.8所示。

Hopper 和 Turton (2001a)[23] 采用 Local Search(LS) 算法来搜索更好的序列。Local Search 算法对序列进行交换或插入操作，若通过修改操作后的序列放置结果更好，则将序列更新并继续对序列进行修改操作。Leung 和 Zhang (2011)[28] 提出的 Fast Layer-Based 算法也通过 Local Search 算法随机寻找两个小矩形并进行交换来生成不同的放置序列。Wei 等人 (2009)[47] 结合了基于轮廓和基于适合度的构造性算法，通过轮廓描述已放置好的小矩形，并评估待放置的小矩形的适合度来选取最适合的小矩形放置到轮廓的拐点上，并采用 Local Search 算法提升解的效果。之后 Wei 等人 (2011)[44] 采用禁忌搜索结合基于轮廓的启发式算法

中国知网 https://www.cnki.net

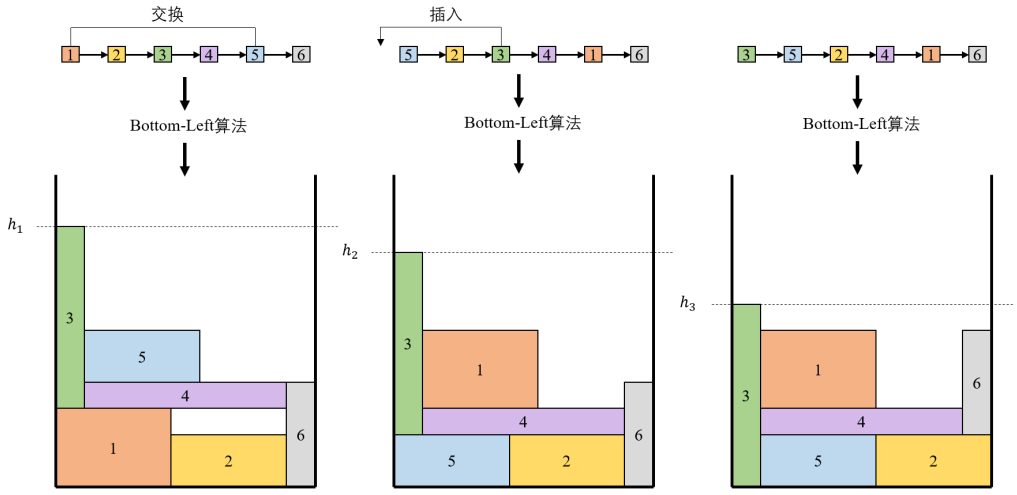


图 1.8 基于序列搜索的启发式算法

求解 2D-SPP。Zhang 等人 (2013)[51] 基于 Randomized Local Search 算法提出了 Binary Search Heuristic 算法来求解 2D-SPP。Chen 等人 (2012)[14] 采用了与 [47] 相同的算法，但是作者在文章中首先对矩形条进行区域划分，并将算法应用在划分出来的区域上。

虽然 Local Search 算法的原理简明，搜索效率高，但搜索容易陷入局部最优解。因此有大量研究采用不同的元启发式算法来取代 Local Search 算法，不同于 Local Search 算法，元启发式算法可以使用某些操作跳出局部最优。Jakobs (1996)[26] 采用遗传算法搜索解空间，生成的解通过 Bottom-Left 算法进行放置。作者将随机序列作为初始群体，通过交叉操作将两个父辈个体从序列位置 i 开始的连续 j 个小矩形进行互换，并选取了三种变异操作以一定概率触发使当前解跳出局部最优。后续有研究采用遗传算法结合不同的构造性算法求解 2D-SPP[10, 18, 22, 38, 41]。

Burke 等人 (2011)[11] 提出了一种全新的基于序列搜索的算法，“Squeaky Wheel”算法。Squeaky Wheel 算法与其它基于序列搜索的算法相同的是都通过修改序列生成新的解，并采用 best-fit 算法来放置小矩形，但不同的是算法引入了惩罚机制。作者首先定义了连续下界，即所有小矩形的面积总和除以矩形条的宽，并给在摆放过程中超过连续下界的小矩形计算惩罚值，惩罚值最高的小矩形插入到下一个序列的首位。

中国知网 https://www.cnki.net

1.3.3.2 基于布局搜索的启发式算法

基于布局搜索的启发式算法与基于序列搜索的不同，基于布局搜索的算法直接在布局上进行修改操作。如图1.9所示，将小矩形 1 和小矩形 2 的摆放位置互换，得到一个更好的解，即高度更低。

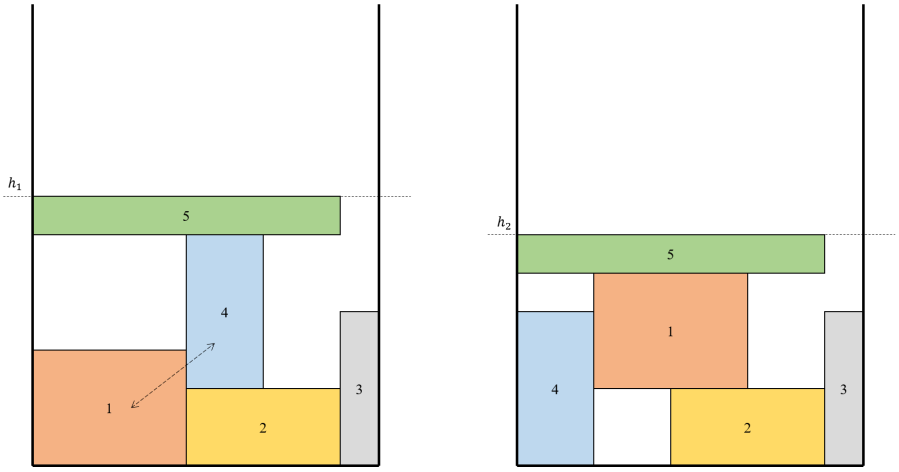


图 1.9 基于布局搜索的启发式算法

基于布局搜索的启发式算法最早由 Alvarez-Valdes 等人 (2005) 提出 GRASP 算法用于求解二维切割问题 [1]，并后续将 GRASP 算法应用到 2D-SPP 上 [2]，算法结合了构造类算法和搜索类算法。构造类算法选择基于适合度的启发式算法，对较小的空余可放置空间赋予较高的优先级，并设计适合度函数对每个小矩形放置的适合度进行评估。搜索类算法提出了四种对布局的修改操作来搜索更优解。Neveu 等人 (2008)[33] 采用与 [2] 相同的修改操作，并选取最高的小矩形放入空白区域并通过构造类算法移动造成重叠的小矩形。

Bortfeldt (2006)[8] 通过启发式算法求解 2D-SPP，算法对层进行搜索，算法通过修改操作对层的序列进行修改，算法返回的结果是层的序列。同样对层进行搜索的研究还有 [46, 52]。

1.3.4 学习类算法

近年来，随着人工智能的兴起，学习类的算法在一些领域取得了巨大的成功。Sutskever 等人 (2014)[40] 提出了基于序列到序列的模型，作者提出的 Sequence to Sequence (Seq2Seq) 模型是一类端到端的算法框架，基本思想就是利用两个 RNN，一个 RNN 作为编码器 (encoder)，另一个 RNN 作为解码器 (decoder)。在编码过

中国知网 https://www.cnki.net

程中，编码器对输入序列进行压缩，生成固定长度的向量，我们称该向量为输入序列通过编码后生成的语义向量 C。在解码过程中，通过编码器生成的语义向量 C 将作为初始状态输入到解码器的 RNN 中，解码器对语义向量 C 进行分析并生成输出序列。该框架推动了学习类算法应用于组合优化问题上的研究。

由于在 Seq2Seq 模型中，原始编解码模型的编码过程会生成一个中间向量 C，用于保存原序列的语义信息。但是这个向量长度是固定的，当输入原序列的长度比较长时，向量 C 无法保存全部的语义信息，上下文语义信息受到了限制，这也限制了模型的理解能力。Bahdanau (2014)[3] 提出了注意力机制来打破这种原始编解码模型对固定向量的限制。注意力机制的原理就是计算当前输入序列与输出向量的匹配程度，匹配度高也就相当于注意力集中，其相对的得分越高，其中注意力计算得到的匹配度权重，只限于当前序列对。

Vinyals (2015)[43] 在 [3, 40] 的基础上，提出了专为组合优化求解而诞生的 Pointer Networks。Pointer Networks (Ptr-Net) 是基于 Seq2Seq 网络并结合简化后的注意力机制生成的一种新的网络架构。Ptr-Net 与 Seq2Seq 类似，都是解决从一个序列到另一个序列的映射问题，不同的是 Ptr-Net 针对的序列问题更加特殊：输出序列的内容与输入序列的内容完全一致，只是序列的顺序发生了改变。这种问题在实际中常见的应用就是组合优化问题，因此 Ptr-Net 首次建立了神经网络与组合优化问题的联系。

Bello (2016)[6] 在 [43] 的基础上，通过强化学习训练 Ptr-Net 并对旅行商问题、背包问题进行求解，得到了比 [43] 更好的实验效果。Hu (2017)[24] 采用 Bello (2016)[6] 的训练方法和 Ptr-Net 的网络框架对一种新的 3D-BPP 问题进行求解，实验结果表明基于强化学习求解 3D-BPP 问题的实验结果相较于启发式算法有 5% 左右的提升。

1.4 主要工作和本文结构

1.4.1 主要工作

如上所述，目前常用于求解 2D-SPP 的算法主要有精确算法和启发式算法。本文提出了一种基于深度强化学习求解二维矩形条装箱问题的算法，通过深度强化学习返回装箱问题的装箱策略，并结合 Bottom-Left-Fill 算法对小矩形进行放置。本文选择基于策略的强化学习方法作为本文的强化学习算法，深度神经网络

中国知网 https://www.cnki.net

模型 Pointer Networks 作为强化学习算法的策略函数，随机性策略作为策略函数的输出，累计奖励的期望值作为目标函数。在训练阶段，深度强化学习通过策略梯度算法计算目标函数的梯度值，并选择 Adam 优化算法更新 Pointer Networks 的模型参数。在测试阶段，策略函数 Pointer Networks 通过 Beam Search 算法搜索装箱策略，策略函数返回的装箱策略进一步结合构造类启发式 Bottom-Left-Fill 算法对小矩形进行装箱。本文最后通过大量实验对基于深度强化学习的 2D-SPP 求解算法进行性能评估，并在 2D-SPP 的经典数据集上对算法进行测试，与启发式搜索算法进行比较。

该算法的优点有：1. 基于线下训练、线上运行，因此对于 2D-SPP 问题的每个实例不需要重新执行算法，只需通过线下训练所得模型简单运算。通过实验表明，在求解大规模的 2D-SPP 时，在装箱策略普遍优于或近似于启发式搜索算法的同时，本文算法的线上运行速度显著优于启发式搜索算法；2. 深度强化学习相较于启发式算法拥有更好的泛化性，启发式算法以问题为导向，根据问题的特殊结构或者性质来改进解，当装箱问题（包括其它组合优化问题）的条件发生细微变化，启发式算法就需针对其变化进行修改。而机器学习类算法通过数据驱动，通过数据自行寻找问题内部的“启发式”，当问题发生细微变化时算法本身不需要进行太多修改；3. 深度强化学习相较于监督学习更适合求解组合优化问题，监督学习需要大量带有正确标签的数据，当 2D-SPP 规模增大时，为上百万的训练样本提供正确标签（最优解）或优质标签（近似最优解）是不现实的；4. 机器学习类算法在组合优化问题上的研究目前还处于探索阶段，目前暂未有通过深度强化学习求解二维矩形条装箱问题的研究，对该问题的探索有助于机器学习类算法在组合优化问题上的研究的进一步发展。

该算法的缺点有：1. 在求解小规模的装箱问题时，启发式搜索算法的线上运行速度同样不错，而本文算法的线下训练时间花费较大；2. 对设备要求较高。

1.4.2 本文结构

本文第一章主要介绍了装箱问题的应用背景，研究意义和具体的研究问题，并总结了 2D-SPP 的研究现状和经典算法，最后介绍了本文的总体框架。

本文第二章主要介绍了深度强化学习算法的深度学习部分。通过循环神经网络、Seq2Seq 模型和注意力机制的描述，对本文所用的深度神经网络模型 Pointer Networks 及其特点进行了详细的介绍，最后介绍了在模型测试阶段使用的 Beam

中国知网 https://www.cnki.net

Search 算法。

本文第三章主要介绍了深度强化学习算法的强化学习部分。首先对强化学习进行了概述，并介绍了本文所用的基于策略的强化学习方法，包括策略函数和奖励函数的设计等。随后介绍了算法优化过程中的策略梯度算法以及梯度计算的数学推导过程，并介绍了实际操作中可能遇到的问题以及解决方法。最后给出了算法优化过程的算法流程图，并对所使用的 Adam 梯度更新算法进行简单介绍。

本文第四章主要通过实验对本文算法的性能进行评估，对算法的可行性与稳定性进行了分析。并通过 2D-SPP 的经典数据集 C 对算法进行测试，与启发式搜索算法在算法实验结果和算法线上运行速度等多方面进行比较，最后总结了本文算法的优缺点。

1.5 本章小结

本章详细介绍了装箱问题的应用背景，研究意义和具体的研究问题，并总结了 2D-SPP 的研究现状和经典算法，最后介绍了本文的总体结构。

中国知网 https://www.cnki.net

中国知网 https://www.cnki.net

第二章　深度神经网络模型

深度强化学习 (Deep Reinforcement Learning, DRL) 是深度学习与强化学习相结合的产物，它兼具了深度学习在感知类问题上强大的理解能力，和强化学习出众的决策能力，并实现了通过端到端的学习。深度强化学习在诸多领域取得成功，也让强化学习理论在实际工业场景中拥有巨大的应用价值。

对于强化学习来说，很多实际应用问题的输入数据是高维的，如图像，声音，算法要根据它们来选择一个动作执行以达到某一预期的目标。比如，对于自动驾驶算法，要根据当前的画面决定汽车的行驶方向和速度。经典的强化学习算法如 Q 学习需要列举出所有可能的情况（称为状态，这是对当前所处环境的抽象），然后进行迭代。Q 学习的经典实现是列举出所有的状态和动作，构建 Q 函数表，这是一个二维的表，然后迭代计算各种状态下执行各种动作的预期回报的最大值。对于高维的输入数据，显然是不现实的，因为如果直接以原始数据作为状态，维数太高，而且状态数量太多。用一个函数来逼近价值函数或策略函数成为解决这个问题的一种思路，函数的输入是原始的状态数据，函数的输出是价值函数值或策略函数值。深度强化学习的基本思想就是通过深度神经网络模型来拟合强化学习中的价值函数和策略函数。本章首先介绍本文所用的深度神经网络模型。

2.1　循环神经网络

在传统的神经网络中，我们认为所有的输入（和输出）彼此之间是相互独立的，如图2.1所示。但是对于很多任务而言，这个观点并不合适。在很多实际应用场景下，序列信息的各个部分是相互关联的，因此对处理序列信息的要求也更高。例如，在我们进行语言交流时，简单地分析单句中各个词的意思显然是不可行的，我们需要对所有词连接起来的完整语句进行分析；在我们观看视频时，仅单独理解每一帧的图像并不能完成对整个视频的理解，我们需将所有帧的画面连接起来进行分析；当我们在求解二维装箱问题时，我们在选择下一个进行放置的小矩形时不能只考虑待放置的小矩形的信息，还要分析已放置好的小矩形的布局。循环神经网络 (Recurrent Neural Network, RNN) 就是为了解决类似上述这

中国知网　https://www.cnki.net

样的问题，相比于传统的神经网络，循环神经网络能够更好的处理序列的信息。

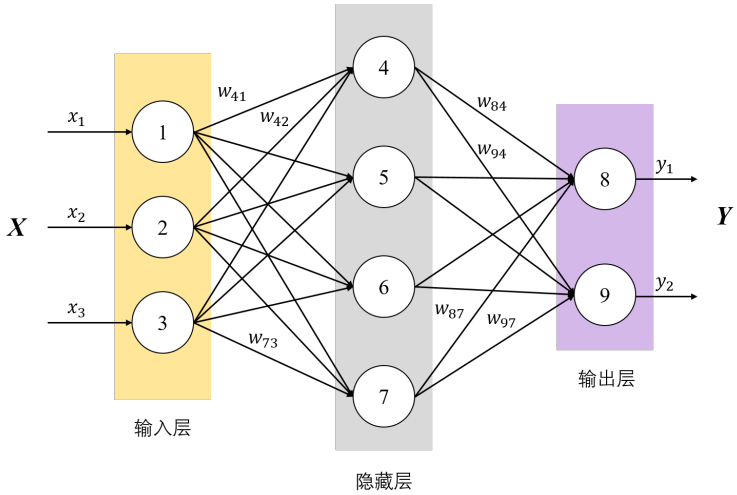


图 2.1 传统神经网络结构图

2.1.1 RNN 模型结构

循环神经网络是一种特殊的神经网络结构。它的特殊之处在于处理序列时，之前分析过的序列信息并不是无用的，而是被网络所“记忆”并应用于之后序列信息输出的计算过程中，即隐藏层中的神经元之间并不是相互独立的。图2.1所示为传统神经网络结构图，其中隐藏层中神经元之间相互独立。RNN 中隐藏层的输入不仅包括输入层的输出，同时还包括上一时刻隐藏层的输出。当前时刻 RNN 的隐层单元 h 不仅仅取决于该时刻的输入 x，还受到上一时刻的隐层单元 h 的影响，如图2.2所示。

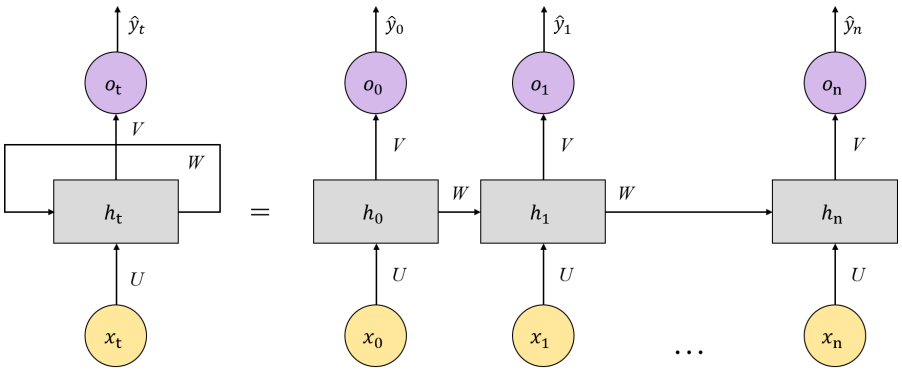


图 2.2 循环神经网络结构图

其中，t 是时刻，x 是输入，h 是隐层单元，o 是输出，$\hat{y}$ 是模型预测值，矩阵

中国知网 https://www.cnki.net

W 就是上一时刻的隐层单元的值作为当前时刻的输入的权重。矩阵 W、U 和 V 是权重共享的，即在每个时刻 W、U 和 V 都是不变的。在循环神经网络中，第一层为输入层，从输入层到隐藏层的过程与传统的神经网络类似。但是隐藏层与传统的神经网络不同，除当前时刻输入层的信息输入 x_t 外，还接受前一时刻的隐藏层的隐层单元 h_{t-1}，在 t 时刻隐层单元 h_t：

$$h_t = f(Ux_t + Wh_{t-1}) \tag{2.1}$$

其中函数 f 通常是非线性函数，常用函数有 $tanh$、$relu$ 和 $sigmoid$ 等。t 时刻的输出 o_t：

$$o_t = Vh_t \tag{2.2}$$

t 时刻的模型预测值 $\hat{y}_t$：

$$\hat{y}_t = g(o_t) \tag{2.3}$$

一般情况下，循环神经网络的任务都是在输出层做分类或者是统计每个词的概率大小，这种情况函数 g 一般选择 $softmax$ 函数。$softmax$ 函数常用于多分类过程，它将多个神经元的输出，映射到 $(0,1)$ 区间内，将其转化成概率分布从而进行分类，假设向量 o_t 的长度为 K，$softmax$ 函数具体公式如下：

$$P(i) = \frac{exp(o_{ti})}{\sum_{i=1}^{K} exp(o_{ti})} \tag{2.4}$$

根据公式2.4可知：$0 < P(i) < 1$，且 $\sum_{i=1}^{K} P(i) = 1$。

2.1.2 反向传播算法

在监督学习中训练数据包含特征 x 和标签 y，神经网络通过训练不断更新网络参数寻找特征与标签之间的联系，在面对只有特征没有标签的数据时，便可以判断出标签。在循环神经网络训练过程，通过反向传播算法 BPTT (Back-Propagation Through Time) 更新神经网络中的权重参数 W、U 和 V。训练数据作为循环神经网络的输入，通过神经网络前向传播得到预测值，每个时刻 t 的预测值都会与训练数据的标签产生一个误差 $e_t = l(y_t - \hat{y}_t)$，总误差可表示为

中国知网 https://www.cnki.net

$E = \sum_t e_t = \sum_t l(y_t - \hat{y}_t)$。反向传播算法 BPTT 的思路就是通过误差值反向传递，运用梯度下降法更新权重参数，也就是求权重参数 W、U 和 V 的梯度:

$$E = \sum_t e_t \tag{2.5}$$

$$\nabla U = \frac{\partial E}{\partial U} = \sum_t \frac{\partial e_t}{\partial U} \tag{2.6}$$

$$\nabla V = \frac{\partial E}{\partial V} = \sum_t \frac{\partial e_t}{\partial V} \tag{2.7}$$

$$\nabla W = \frac{\partial E}{\partial W} = \sum_t \frac{\partial e_t}{\partial W} \tag{2.8}$$

根据公式2.6~2.8可知，权重参数 W、U 和 V 的梯度是每个时刻的误差的偏导数之和。这里以权重参数 W 在 $t = 3$ 时刻的误差 e_3 的偏导计算为例。根据链式求导法可以得到 $t = 3$ 时刻的偏导数为:

$$\frac{\partial e_3}{\partial W} = \frac{\partial e_3}{\partial o_3} \cdot \frac{\partial o_3}{\partial h_3} \cdot \frac{\partial h_3}{\partial W} \tag{2.9}$$

根据公式2.1可知，权重参数 W 除了与 h_3 有关, 还和前一时刻的隐层单元 h_2 有关，对 h_3 展开可得：

$$\frac{\partial h_3}{\partial W} = \frac{\partial h_3}{\partial h_3} \cdot \frac{\partial^+ h_3}{\partial W} + \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial W} \tag{2.10}$$

其中，$\frac{\partial^+ h_3}{\partial W}$ 表示 h_3 对 W 直接求导，而不考虑 h_2 的因素。对 h_2 展开可得：

$$\frac{\partial h_2}{\partial W} = \frac{\partial h_2}{\partial h_2} \cdot \frac{\partial^+ h_2}{\partial W} + \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial W} \tag{2.11}$$

对 h_1 展开可得：

$$\frac{\partial h_1}{\partial W} = \frac{\partial h_1}{\partial h_1} \cdot \frac{\partial^+ h_1}{\partial W} + \frac{\partial h_1}{\partial h_0} \cdot \frac{\partial h_0}{\partial W} \tag{2.12}$$

将上述公式2.10~2.12合并可得：

$$\frac{\partial h_3}{\partial W} = \sum_{k=0}^{3} \frac{\partial h_3}{\partial h_k} \cdot \frac{\partial^+ h_k}{\partial W} \tag{2.13}$$

中国知网 https://www.cnki.net

将公式2.13带入公式2.9中，可得权重参数 W 在 $t=3$ 时刻的误差 e_3 的偏导为：

$$\frac{\partial e_3}{\partial W}=\sum_{k=0}^{3}\frac{\partial e_3}{\partial o_3}\cdot\frac{\partial o_3}{\partial h_3}\cdot\frac{\partial h_3}{\partial h_k}\cdot\frac{\partial^{+}h_k}{\partial W} \tag{2.14}$$

2.1.3 LSTM 模型

相比于传统的神经网络，循环神经网络能够更好的处理序列的信息。而普通的循环神经网络在处理较长的序列数据时，由于序列数据较长，序列数据中相关信息和当前预测位置之间的间隔就会变大，这个间隔不断增大时，RNN 会丧失学习到距离如此远的信息的能力。在理论上，RNN 可以处理这样的“长距离依赖”问题，可以通过仔细挑选参数来解决这类问题中的最初级形式。但在实践中，RNN 往往不能够成功学习到这些远距离的信息，序列过长会导致优化时出现梯度消散的问题。

Long Short-Term Memory (LSTM)，是一种特殊的 RNN 网络，该网络正是为了解决“长距离依赖”问题而设计。LSTM 由 Hochreiter 和 Schmidhuber (1997)[20]提出，后续并有许多学者对其进行了改进和普及。他们的工作被用来解决了各种各样的问题，直到目前还被广泛应用。LSTM 作为一种特殊的 RNN 模型，与标准 RNN 模型中重复的模块包含单一的层不同，例如一个 tanh 层。LSTM 重复的模块拥有一个不同的结构，通过四个网络层进行交互，如图2.3所示。

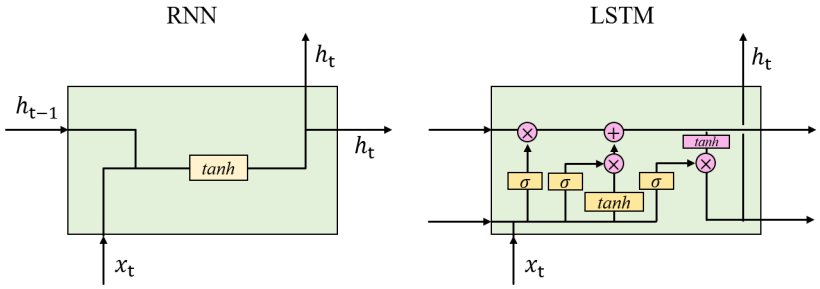


图 2.3 标准 RNN 模型和 LSTM 模型

LSTM 的具体模型结构图如图2.4所示。其中，输入门、遗忘门、输出门是 LSTM 模型主要的组成结构，利用三个门的结构组成来保存与控制流经 LSTM 的信息。在 t 时刻，LSTM 模型的输入有三个：$t-1$ 时刻的细胞状态 c_{t-1}、$t-1$ 时刻的隐层状态 h_{t-1}、t 时刻的输入值 x_t。同时，在 t 时刻，LSTM 模型的输出有两个：t 时刻的细胞状态 c_t、t 时刻的隐层状态 h_t。其中，模型中的 x，h，c 都是向量。三个门层结构在 t 时刻的具体运作首先通过遗忘门 f_t 决定将从 $t-1$ 时

中国知网 https://www.cnki.net

刻的细胞状态 c_{t-1} 中丢弃多少信息。f_t 会读取 h_{t-1} 与 x_t，并输出一个在 0 到 1 之间的数值给细胞状态 c_{t-1}，1 代表细胞状态 c_{t-1} “完全保留”，0 代表细胞状态 c_{t-1} “完全舍弃”：

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{2.15}$$

输入门 i_t 记忆现在的某些信息，信息通过 $tanh$ 函数激活，并通过 $sigmoid$ 控制门遗忘一部分内容：

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{2.16}$$

$$\widetilde{C}_t = tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \tag{2.17}$$

将过去与现在的记忆进行合并：

$$C_t = f_t * C_{t-1} + i_t * \widetilde{C}_t \tag{2.18}$$

输出门 o_t 通过 $sigmoid$ 函数用于控制细胞状态 c_{t-1} 对 h_t 的影响程度：

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \tag{2.19}$$

$$h_t = o_t * tanh(C_t) \tag{2.20}$$

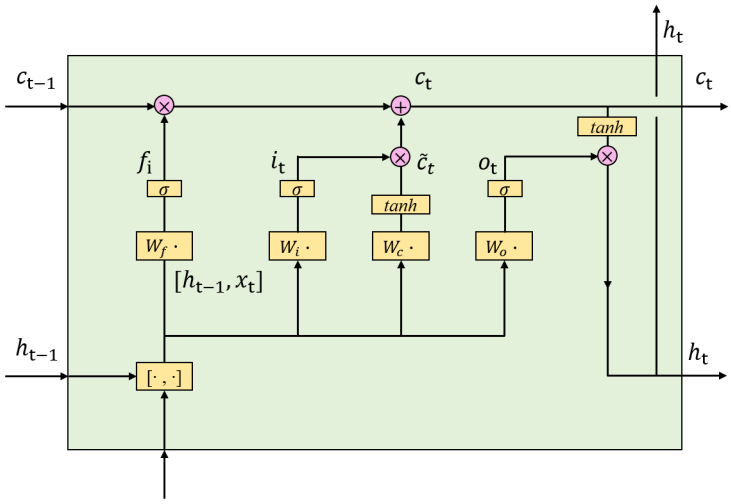


图 2.4 LSTM 结构图

中国知网 https://www.cnki.net

2.2 Sequence to Sequence 模型

2.2.1 Seq2Seq

对于序列生成问题而言，目前最常用的模型是从序列到序列的模型框架，即 Sequence to Sequence 模型，简称 Seq2Seq。

最基础的 Seq2Seq 模型包含了三个部分，即编码器 (Encoder)、解码器 (Decoder) 以及连接两者的中间状态向量 C，编码器将输入向量编码成固定长度的中间状态向量 C 并将 C 传给解码器，解码器对中间状态向量 C 进行分析解码并输出结果，如图2.5所示。其中，编码器和解码器中都应用了 LSTM 作为神经元（注：编码器和解码器中神经元不局限于 LSTM，可以是标准 RNN 或结构更为复杂的神经元）。编码器中神经元在第一个时间步输入 x_0，第二个时间步输入 x_1，以此类推，在最后一个时间步的隐层状态就是中间状态向量 C，它将作为解码器中神经元的初始状态。然后在解码器神经元中，每个时间步的输出将会作为下一个时间步的输入。以此类推，直到神经元在某个时间步预测输出特殊符号 <END> 表示结束。中间状态向量 C 可以看作是所有的输入序列信息的一个集合，所有的输入序列的信息都包括在 C 里面。

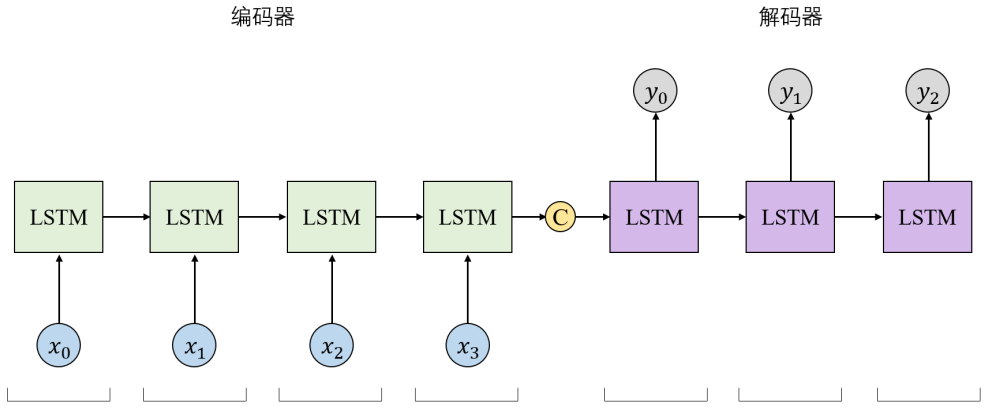


图 2.5 Sequence to Sequence 模型

在 Seq2Seq 模型中我们的训练数据是由形式为 $< X, Y >$ 的序列数据集合构成，如下表示：

$$X =< x_1, x_2, ..., x_t > \tag{2.21}$$

$$Y =< y_1, y_2, ..., y_k > \tag{2.22}$$

其中 t，k 表示序列 X 和 Y 的长度。编码器对输入的序列 X 进行编码，通过一

中国知网 https://www.cnki.net

系列非线性变换得到中间状态向量 C:

$$C = h(x_1, x_2, ..., x_t) \tag{2.23}$$

解码器将中间状态向量 C 结合之前时刻的输出进行解码，通过一系列非线性变换生成下一时刻的输出：

$$y_i = j(C, y_1, y_2, ..., y_{i-1}) \tag{2.24}$$

按照以上过程依次进行，直到解码到限制最大字符长度或者解码出结束字符 <END> 为止，我们便得到了解码器输出的结果序列。值得注意的是，解码器的每一个输出 y_i 对应的词表中每个词的概率大小，并选择概率最高的词作为输出。

2.2.2 模型缺陷

Seq2Seq 模型常用于翻译系统和对话生成领域，通过大量的语料样本训练语言模型，模型可以对当前语句进行分析并预测后续的字词，模型对文本预测的过程实则是对一个固定大小的词表中的所有词汇进行概率预测并选择概率最大的词汇进行输出。

Seq2Seq 模型实现了从端到端的序列模型，但是 Seq2Seq 模型并不能够很好地求解 2D-SPP。因为 Seq2Seq 模型的词表是事先固定的，比如将 Seq2Seq 模型应用于中译英的翻译系统，词表就是所有的英文单词，是事先固定的。在翻译过程中，编码器的输入就是中文句子，解码器的每一个输出 y_i 对应的就是词表中所有英文单词的概率分布，并从中选取概率最大的单词进行输出。词表不会因为输入的中文句子的长度和内容的不同而发生变化。

而在求解 2D-SPP 时，词表的长度取决于问题输入序列的长度，即待放置的小矩形的总数。在求解规模为 30 的 2D-SPP 时，模型输入就是长度为 30 的序列，序列中每个元素是对应小矩形的长和宽，词表就是 30 个小矩形的序号，词表长度是 30，解码器的每一个输出 y_i 对应的就是这 30 个小矩形的概率分布，并从中选取概率最大的小矩形进行输出。而当求解规模为 50 的 2D-SPP 时，模型输入序列的长度为 50，词表的长度也相应变成 50。因此，在求解 2D-SPP 时，词表根据输入的序列长度的变化而发生变化，并非事先固定的。因此，应用 Seq2Seq 模型求解 2D-SPP 受到了限制。我们将 2D-SPP 这类问题称之为“输出严重依赖输

中国知网 https://www.cnki.net

入”问题。不仅只有 2D-SPP，许多常见的组合优化问题都是“输出严重依赖输入”问题，如旅行商问题、凸包问题等。

2.3 Pointer Networks 模型

在 Sequence to Sequence 模型的基础上，Vinyals (2015)[43] 做了相应改进提出了 Pointer Networks 模型，该模型通过引入特殊的 Attention 机制来解决“输出严重依赖输入”问题。

2.3.1 传统的 Attention 机制

在 Seq2Seq 模型中，解码器中的神经元在每一个时刻通过 $s_t = f(s_{t-1}, x_t, C)$ 计算隐层状态 s_t，f 代表非线性函数，如图2.5中 f 代表 LSTM，C 为编码器输出的中间状态向量。可见，解码器在每个时刻计算 s_t 时都使用同一个状态向量 C，而在解码器不同时刻所需要重点提取的编码器中的信息是不同的。因此，Bahdanau (2014)[3] 提出了注意力机制, 打破 Seq2Seq 模型对固定向量 C 的限制。注意力机制的原理就是计算当前输入序列与输出向量的匹配程度，匹配度高也就是注意力集中点其相对的得分越高，其中注意力计算得到的匹配度权重，只限于当前序列对。注意力机制的提出是从日常生活中获得启发，当人类在阅读长句子时，会重点关注长句子的某几个重要信息部位，在这些区域集中更多的注意力，同时略读一些无用信息部位。

注意力机制的实现思想是编码器在每个时刻 t 计算隐层单元 s_t 之前，为该时刻 t 重新计算中间状态向量 C_t，则 $s_t = f(s_{t-1}, x_t, C_t)$。首先定义编码器中的隐层状态为 $(h_1, h_2, ..., h_n)$，解码器的隐层状态定义为 $(s_1, s_2, ..., s_m)$，解码器中时刻 t 的中间状态向量 C_t 通过对编码器中隐层状态 $(h_1, h_2, ..., h_n)$ 加权求和计算所得。

$$C_t = \sum_{j=1}^{n} \alpha_{tj} h_j \tag{2.25}$$

其中，权值 α_{tj} 由 e_{tj} 经过 $softmax$ 函数计算所得:

$$\alpha_{tj} = \frac{exp(e_{tj})}{\sum_{k=1}^{n} exp(e_{tk})}, \ j \in \{1, 2, ..., n\} \tag{2.26}$$

其中，$e_{tj} = a(s_{t-1}, h_j)$，函数 a 表示 s_{t-1} 和 h_j 的相似程度，因此被称为相似度函

中国知网 https://www.cnki.net

数，a 的选取有很多选择，可以选择简单的余弦函数，或是如下的相似度函数：

$$\begin{aligned} e_{tj} &= a(s_{t-1}, h_j) \\ &= v^{\top} tanh(W_1 h_j + W_2 s_{t-1}) \ , \ j \in \{1, 2, ..., n\} \end{aligned} \tag{2.27}$$

其中训练参数 v 为向量，W_1，W_2 为矩阵。

可见，解码器在时刻 t 时，通过将解码器在 $t-1$ 时刻的隐层状态 s_{t-1} 与编码器中每一个隐层状态 $(h_1, h_2, ..., h_n)$ 进行相似度计算得到权值，再与 $(h_1, h_2, ..., h_n)$ 加权求和计算状态向量 C_t，那么与 s_{t-1} 具有较高相似度的编码器隐层状态 h_i 将分配有较大的权值，从而在中间状态向量 C_t 中拥有更多的“注意力”。

2.3.2 Pointer Networks 模型结构

Pointer Networks 模型是在 Seq2Seq 的基础上，结合了改进后的注意力机制。与传统的注意力机制不同，作者对注意力机制进行了简化，将针对输入序列的权重作为指向输入序列的指针，在每次预测一个元素的时候找到输入序列中对应权重最大的那个元素，如图2.6所示。

与传统的注意力机制相同，Pointer Networks 在时刻 t 同样计算解码器隐层状态 $(s_1, s_2, ..., s_m)$ 中的 s_t 与编码器中的隐层状态 $(h_1, h_2, ..., h_n)$ 与的相似度 e_{tj}:

$$e_{tj} = v^{\top} tanh(W_1 h_j + W_2 s_{t-1}) \ , \ j \in \{1, 2, ..., n\} \tag{2.28}$$

e_t 可表示为 $e_t = [e_{t1}, ..., e_{tn}]$。与传统的注意力机制不同的是，Pointer Networks 直接将 e_t 经过 $softmax$ 规范化后得到的概率分布作为基于输入序列的输出预测分布：

$$p(y_t | x_1, x_2, ..., x_n, \theta, y_1, ..., y_{t-1}) = softmax(e_t) \tag{2.29}$$

Pointer Networks 直接使用 $softmax$ 得到的基于输入序列的概率分布结果，去拷贝编码器对应的输入元素作为的解码器的输出元素。

Pointer Networks 便是 Seq2Seq 结合简化后的注意力机制后得到的模型。Pointer Networks 通过直接选择 s_t 与 $(h_1, h_2, ..., h_n)$ 中相似度最高的 h_i，来“指向”输入序列中的对应元素来作为输出。因此，模型不再是根据固定内容的词表的概率分布来预测下一个输出，而是直接预测输出应该对应输入序列的哪一个元素，从而解决“输出严重依赖输入”问题。

中国知网 https://www.cnki.net

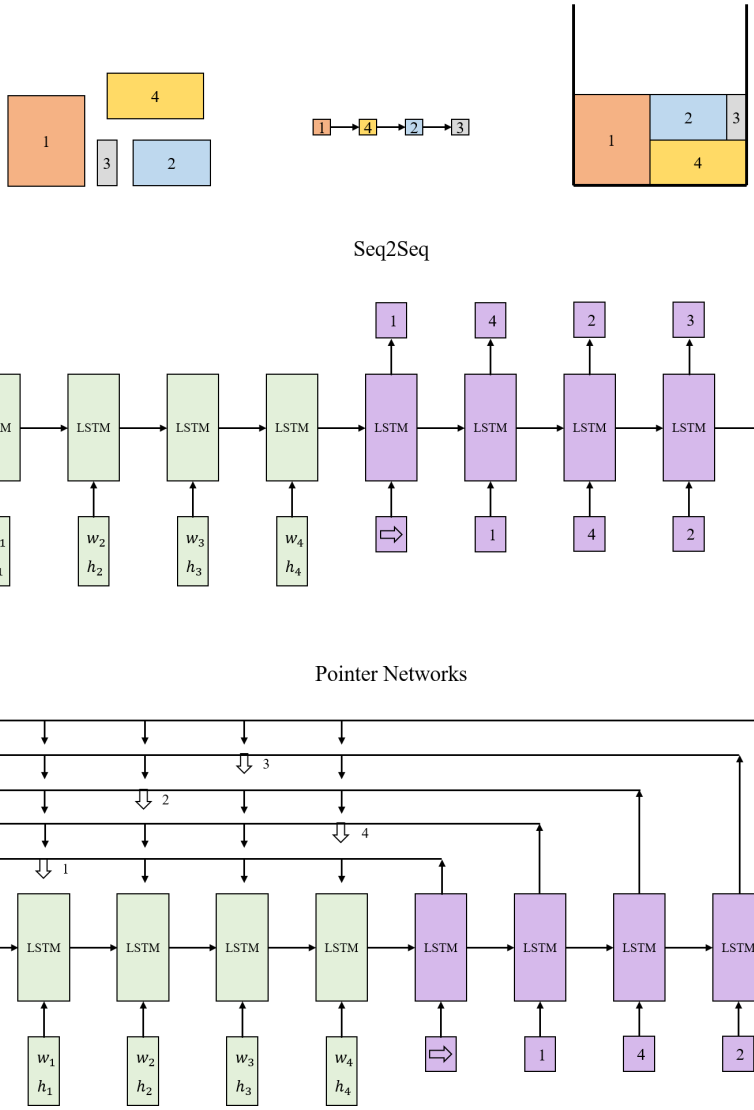


图 2.6 Seq2Seq 和 Pointer Networks 模型

2.4 Beam Search 算法

Beam search 算法是一种启发式搜素算法，在优化领域中属于最佳优先算法。最佳优先算法通过启发式规则衡量计算所得解与目标解之间的“距离”，并按照“距离”对其排序。而与最佳优先算法不同的是，Beam search 算法只保存部分解作为算法的候选解，而不是所有的解，算法通过宽度优先搜索构建搜索树并在每一层通过启发式规则对解进行排序并选择最好的 k 个解作为算法候选解，其中 k 为 Beam search 算法的集束宽度 (beam size)。最好的 k 个解可以在搜索树中继续向下进行搜索，当集束宽度越大时，候选解越多，被筛选掉的解越少。因为集束宽度的存在，算法不能保证搜索到全局最优解，因为最优解有可能在之前的搜索

中国知网 https://www.cnki.net

过程中被筛选掉了。

Beam search 算法通常在测试的时候使用，因为在训练过程中，解码器每个时刻的输出是有与之对应的正确答案做参照，也就不需要 Beam search 算法去提高输出的准确率。现在以图2.6的 2D-SPP 问题为例。首先，编码器对小矩形序列 $\{(w_1,h_1),(w_2,h_2),(w_3,h_3),(w_4,h_4)\}$ 进行编码，得到中间状态向量 C。解码器得到中间状态向量 C 后，对其进行解码，依次输出成放置顺序。在正式解码之前，我们首先设置 Beam search 算法中的 beam size=2，这表示选择对应概率最大的两个元素。

首先我们讨论解码器的第一个时刻的解码 y_1，在给定中间状态向量 C 的情况下，首先选择小矩形序列中可能最大的两个，也就是依次选择条件概率 $P(y_1|C)$ 最大和次大的两个小矩形，比如这里概率最大的两个矩形分别是 $\{(w_1,h_1),(w_4,h_4)\}$。

接下来进行解码器的第二个时刻的解码 y_2，在给定中间状态向量 C 和前一时刻输出 y_1 的情况下，在第二个时刻，各个小矩形作为该时刻输出的条件概率为 $P(y_2|y_1,C)$。由于上一时刻选取了概率最大的两个小矩形 $\{(w_1,h_1),(w_4,h_4)\}$，在 $((w_1,h_1),C)$ 的条件下分别有四种组合，每种组合的概率为 $P((w_1,h_1)|C)\cdot P(y_2|(w_1,h_1),C)$。同理，在 $((w_4,h_4),C)$ 的条件下同样有四种组合，每种组合的概率为 $P((w_4,h_4)|C)\cdot P(y_2|(w_4,h_4),C)$。这样便总共得到了 8 种组合，每一种组合对应概率 $P(y_1|C)\cdot P(y_2|C,y_1)$，接着在这 8 种组合中选择对应概率最高的两种组合。接下来重复上述过程，逐步生成序列，直到遇到结束标识符停止，最终得到两个序列并挑选概率最大的序列作为输出序列。Beam Search 算法的思想如图2.7所示，可见，当 beam size=1 时，Beam Search 算法就变成了贪心算法。

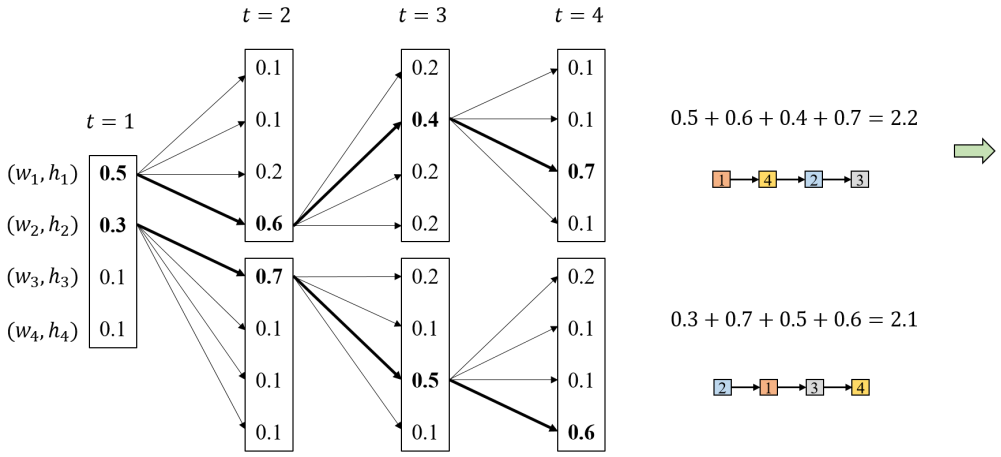


图 2.7 Beam Search 算法

中国知网 https://www.cnki.net

2.5 本章小结

本章主要介绍了本文所使用的深度神经网络模型 Pointer Networks。本章通过对循环神经网络、Seq2Seq 模型和注意力机制的描述，对 Pointer Networks 及其模型特点进行了详细的介绍。本章最后介绍了在测试阶段使用的 Beam Search 算法。

中国知网 https://www.cnki.net

中国知网 https://www.cnki.net

第三章　基于深度强化学习的 2D-SPP 算法

在介绍完本文所用的深度神经网络模型 Pointer Networks 后，本章将介绍通过深度强化学习的方法求解 2D-SPP。最初，有学者通过监督学习的方法运用 Pointer Networks 求解组合优化问题，如旅行商问题、凸包问题等。监督学习的方法通过大量带有特征 x 和标签 y 的训练数据，将 Pointer Networks 输出的概率分布与标签对应的概率分布的交叉熵作为损失函数，运用2.1.2介绍的反向传播算法更新 Pointer Networks 的参数。而运用监督学习的方法求解这类组合优化问题的弊端在于：1. 模型的效果取决于训练样本的标签质量，2. 由于问题是 NP 难的，获取高质量的标签需要很大成本。因此，通过强化学习训练 Pointer Networks 更适合求解 2D-SPP 问题。

3.1　强化学习

强化学习属于机器学习的一种，如图3.1所示。而与其他机器学习方法不同的是：1. 强化学习没有带标签的训练数据，只有奖励信号，2. 强化学习反馈的信号具有滞后性，3. 强化学习中时间（序列）是一个重要因素，4. 强化学习中当前的动作影响后续接收的数据。

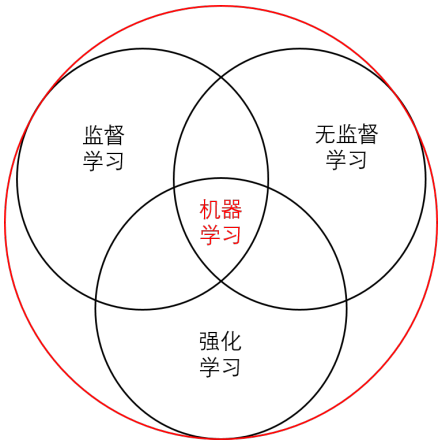


图 3.1 机器学习范畴

强化学习中的重要元素包括：奖励 (Rewards)、智能体 (Agent)、环境 (Environment)、状态 (State) 等。强化学习的本质就是通过与环境的互动来学习如何达成一个目标。这个学习和做决策的主体就叫智能体。与智能体交互的对象就是环

中国知网　https://www.cnki.net

境。智能体会持续地与环境交互，根据当前的状态选择行为，而环境会反馈给智能体新的状态和奖励，如图3.2所示。接下来分别通过奖励、环境与智能体、历史与状态、智能体的组成部分四块进行介绍。

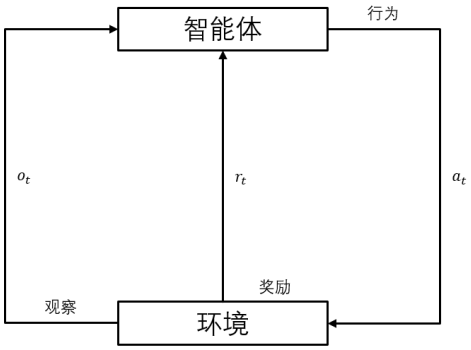


图 3.2 强化学习里智能体和环境的互动

奖励：奖励 r_t 是一个标量反馈信号，表示的是一个智能体在时刻 t 的表现，而智能体主要的工作则是最大化累计的奖励。也就是说智能体通过选择行为 a_t 来最大化总体的未来的奖励。可以看出行为可能有长期的因果关系并且奖励可能延迟，即可能是牺牲即时的奖励来获取更长期的奖励。

环境与智能体：环境和智能体是强化学习的两个重要组成部分，其中智能体接受时刻 $t-1$ 的观察 (Observation) o_{t-1} 和来自环境反馈的奖励 r_{t-1}，然后执行行为 a_t，然后环境接受来自时刻 t 的智能体执行的行为 a_t，并产生下一时刻的观察 o_t 和奖励 r_t，具体过程如图3.2所示。

历史与状态：当智能体和环境进行交互时，会产生一系列的观测、行为、奖励，即历史，$h_t = o_1, a_1, r_1, o_2, a_2, r_2, ..., o_t, a_t, r_t$。智能体要采取什么行为、环境将产生什么观测，并返回什么奖励，完全取决于历史。而状态就是决定将要发生事情的信息，通常状态是历史的函数：$s_t = f(h_t)$。其中，状态也分环境状态和智能体状态。环境状态 s_t^e 是环境的一种私有呈现，它包含了用于下一步产生观测和奖励的全部信息，而这些信息对智能体来说通常是不可见的，即使可见也通常含有很多无关的信息。而智能体状态 s_t^a 是智能体内部的一种呈现, 包含了用于产生下一步动作的全部信息，这些信息就是强化学习算法需要的信息，它可以由历史的某种函数所表示：$s_t^a = f(h_t)$。s_t^e 和 h_t 都具有马尔科夫性质 (Markov)，马尔科夫性质如定义3.1所述。当环境是全部可观测时，即：$s_t^a = s_t^e = o_t$，整个过程也就变成了一个马尔科夫决策过程。在 2D-SPP 中，环境便是完全可观测的。

中国知网 https://www.cnki.net

定义 3.1 状态 S_t 具有马尔可夫性质当且仅当:

$$P[S_{t+1}|S_t] = P[S_{t+1}|S_1, ..., S_t] \tag{3.1}$$

智能体的组成部分：强化学习中的智能体由以下三部分的一个或者多个构成：策略 (Policy)、值函数 (Value Function)、模型 (Model)。策略定义了智能体的行为方式，是从状态 s_t 到行为 a_t 的映射，策略分为确定性策略：$a_t = \pi(s_t)$ 和随机性策略:$\pi(a|s) = P[a_t = a|s_t = s]$，策略可以是一个函数，也可以是一个查询表。值函数是一个对未来奖励的预测，用来评价当前状态 s_t 的好坏程度：$v_\pi(s) = E[r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + ...|s_t = s]$。模型通常用于预测环境的变化，帮助智能体决定下一步的行为，模型需要预测下一个可能状态发生的概率：$P^a_{ss'} = P[s_{t+1} = s'|s_t = s, a_t = a]$ 以及预测可能获得的即时奖励：$r^a_{ss'} = E[r_{t+1}|s_t = s, a_t = a]$，但模型不是构成智能体的必须组成部分。

以上四个部分简述了强化学习中的各个重要元素的组成和作用。而根据智能体的组成可以将强化学习算法分成五大类：Policy Based、Value Based、Actor Critic、Model Free、Model Based，如图3.3所示。本文运用的正是 Policy Based 深度强化学习方法。

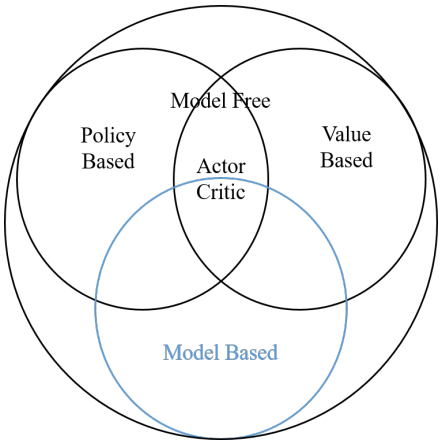


图 3.3 强化学习方法分类

3.2 Policy Based 深度强化学习算法

Policy Based 强化学习方法具有以下优点：1.Policy Based 强化学习方法能够学习到随机策略，而 Value Based 方法通常无法学习到随机策略，2.Policy Based 强化学习方法在策略梯度的计算上相比 Value Based 方法更为简单，3.Policy Based

中国知网 https://www.cnki.net

强化学习方法通常具有更好的收敛性，因为学习过程中虽然每次只做微小改变，但总往好的方向改善，4. 对于具有高维度和连续状态空间的问题来说，Policy Based 强化学习方法更加高效。同时，Policy Based 强化学习方法的不足之处有：1. 优化过程有时朝着梯度方向改变的增量会有较高的变异性，会影响算法速度，2. 容易陷入局部最优解。

3.2.1 策略函数设计

Policy Based 深度强化学习算法直接有一个参数化的策略函数，通过计算累计奖励期望值对策略函数参数的梯度来不断更新策略函数中的参数。本文采用 Pointer Networks 作为强化学习的策略函数，如图2.6所示。

假设参数化的策略函数为 $\pi(s,\theta)$，参数 θ 包含了 Pointer Networks 模型的所有参数：$\theta = \{w_1, w_2, ..., b_1, ...\}$，$\pi$ 的输入为状态 s，输出为行为 a。如3.1所述，策略函数分为随机性策略函数和确定性策略函数。在本问题中，我们的策略函数是随机性的，即策略函数返回的不是一个具体的行为 a，而是所有可采取的行为对应的一个概率分布，并随机选取其中的一个行为进行输出。例如：假设有 4 个小矩形 $\{a_1, a_2, a_3, a_4\}$ 要放置入矩形条中，在时刻 t，策略函数 $\pi(s_t,\theta)$ 输入状态 s_t，随机性策略函数的输出为这 4 个小矩形的概率分布：$\pi(a_i|s_t,\theta) = p(a_t = a_i|s_t,\theta), i = 1,2,3,4$，如 $\pi(a|s_t,\theta) = \{0.1, 0.2, 0.3, 0.4\}$，随机性策略函数根据概率分布 $\pi(a|s_t,\theta)$，依概率随机选取行为 a_i，而非确定性地选取对应概率最大的行为 a，如图3.4所示。

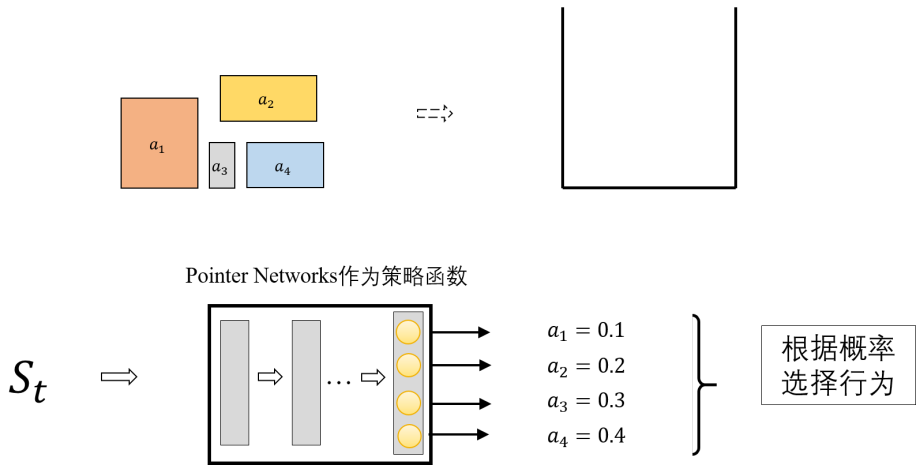


图 3.4 随机性 Policy 函数

中国知网 https://www.cnki.net

3.2.2 奖励函数设计

3.2.2.1 2D-SPP 描述

本章介绍通过 Policy Based 深度强化学习求解二维矩形条装箱问题。二维矩形条装箱问题描述如下：矩形条的宽度 W 固定，高度 H 不固定，给定 n 个小矩形 $\{(w_1,h_1),(w_2,h_2),...,(w_n,h_n)\}$，目标是将所有小矩形装下，且小矩形不能旋转，小矩形之间不能重叠，并最小化所使用的矩形条的高度 H。每个小矩形的宽度为 w_i，高度为 h_i，$1 \leq i \leq n$，$0 \leq h_i$，$0 < w_i \leq W$，如图3.5所示，H 即为将图左的十个小矩形装入宽为 W 的矩形条后的高度。

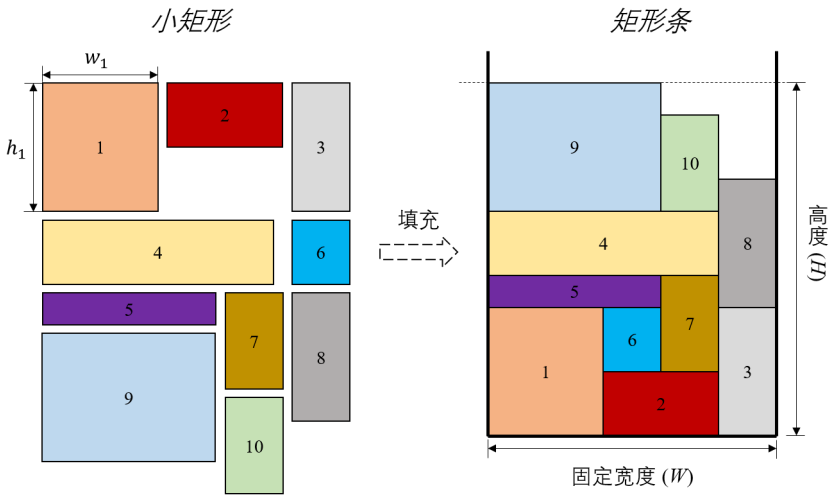


图 3.5 二维矩形条装箱问题

3.2.2.2 奖励函数

如3.1所述，在 2D-SPP 中，环境是完全可观测的，即 $s_t^a = s_t^e = o_t$，整个过程也就变成了马尔可夫决策过程。假设 2D-SPP 的小矩形数量规模为 T，则一个完整的马尔可夫决策过程对应的的采样轨迹 τ 如图3.6所示。

如图3.6所示，我们有：

$$p_\theta(\tau) = p(s_1)\prod_{t=1}^{T} p_\theta(a_t|s_t)p(s_{t+1}|s_t,a_t) \tag{3.2}$$

其中，θ 表示 Pointer Networks 的模型参数，式子3.2中用到了马氏性，即认为一个状态只与与之相邻的前一个状态有关。在得到采样轨迹 τ 后，便可计算 τ 的总

中国知网 https://www.cnki.net

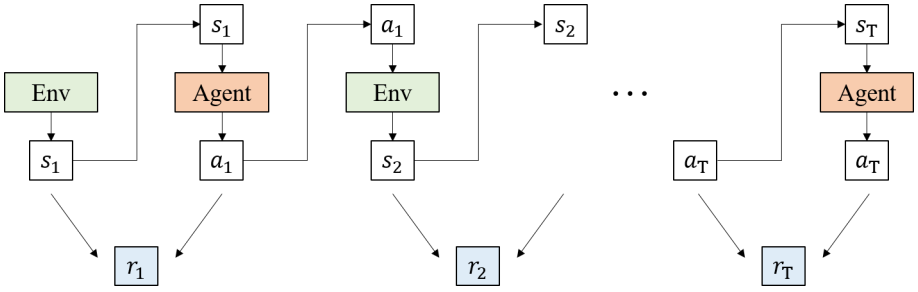


图 3.6 采样轨迹 τ

奖励值：

$$R(\tau) = \sum_{t=1}^{T} r_t \tag{3.3}$$

在求解 2D-SPP 时，Pointer Networks 返回的采样轨迹 τ 中给出了小矩形放置顺序 $\{a_1, a_2, ..., a_T\}$，我们通过 Bottom-Left-Fill 算法对小矩形进行放置并计算最终高度 H，并取最终高度 H 的倒数作为该采样轨迹的总奖励值：$R(\tau) = \frac{1}{H}$。

如3.2.1所述，本文采用的是随机性策略函数。因此，在完全相同的 Pointer Networks 结构以及完全相同的模型参数 θ 之下，每次生成的采样轨迹 τ 都是不同的。本文深度强化学习目标最大化的奖励函数并不是计算单条采样轨迹在的某个时刻返回的小奖励值，也不是计算单条采样轨迹的总奖励值，而是计算 Pointer Networks 在当前模型参数 θ 下生成采样轨迹 τ 的总奖励值的期望值：

$$\bar{R}_\theta = \sum_{\tau} R(\tau) p_\theta(\tau) = E_{\tau \sim p_\theta(\tau)}[R(\tau)] \tag{3.4}$$

如上，$\bar{R}_\theta$ 便是奖励函数，模型训练过程的目标则是通过更新模型参数 θ 来优化 $\bar{R}_\theta$，使之最大化即可。本文采取策略梯度算法对 Pointer Networks 的模型参数进行优化。

3.3 算法优化

3.3.1 策略梯度算法

策略梯度算法是强化学习中最常用的优化算法，通过目标函数 $\bar{R}_\theta$ 对模型参数 θ 计算梯度，从而对 Pointer Networks 模型参数进行更新，寻找一组可以最大

中国知网 https://www.cnki.net

化 $\bar{R}_\theta$ 的模型参数 θ^*：

$$\theta^* = \underset{\theta}{argmax}(\bar{R}_\theta) \tag{3.5}$$

如3.2.2.2所述，Pointer Networks 在当前模型参数 θ 下生成采样轨迹 τ 的总奖励值的期望值为：

$$\bar{R}_\theta = \sum_\tau R(\tau)p_\theta(\tau) = E_{\tau\sim p_\theta(\tau)}[R(\tau)] \tag{3.6}$$

其中，$R(\tau)$ 是一个随机变量，描述了不同采样轨迹的总奖励值。$p_\theta(\tau)$ 表示在当前策略函数下出现各种采样轨迹的概率分布，与策略函数有关。

对 $\bar{R}_\theta$ 求导：

$$\begin{aligned}
\nabla_\theta \bar{R}_\theta &= \nabla_\theta \sum_\tau R(\tau)p_\theta(\tau) \\
&= \sum_\tau \nabla_\theta p_\theta(\tau)R(\tau) \\
&= \sum_\tau p_\theta(\tau)\frac{\nabla_\theta p_\theta(\tau)}{p_\theta(\tau)}R(\tau) \\
&= \sum_\tau p_\theta(\tau)\nabla_\theta log p_\theta(\tau)R(\tau) \\
&= E_{\tau\sim p_\theta(\tau)}[\nabla_\theta log p_\theta(\tau)R(\tau)]
\end{aligned} \tag{3.7}$$

期望可以通过大量样本求平均的方式进行估计：

$$\begin{aligned}
\nabla_\theta \bar{R}_\theta &= E_{\tau\sim p_\theta(\tau)}[\nabla_\theta log p_\theta(\tau)R(\tau)] \\
&= \frac{1}{N}\sum_{i=1}^{N}[\nabla_\theta log p_\theta(\tau_i)R(\tau_i)]
\end{aligned} \tag{3.8}$$

其中，$\nabla_\theta log p_\theta(\tau_i)$ 梯度值是 $p_\theta(\tau_i)$ 在当前参数 θ 下出现概率变化最陡的方向（log 是单调递增函数，因此等同于求是 $p_\theta(\tau_i)$）。梯度正方向是 $p_\theta(\tau_i)$ 增大最快的方向，反方向是减小最快的方向。$p_\theta(\tau_i)$ 越大表示采样轨迹 τ_i 出现的概率越大，而 $R(\tau_i)$ 表示其总奖励值，我们希望总奖励值越大的采样轨迹出现的概率越大。式3.8中的 $R(\tau_i)$ 通过样本统计进行估计，接下来介绍 $p_\theta(\tau_i)$ 如何计算。

假设采样轨迹 $\tau_i = \{s_{i,1}, a_{i,1}, r_{i,1}, s_{i,2}, a_{i,2}, r_{i,2}, ..., s_{i,T}, a_{i,T}, r_{i,T}\}$，采样轨迹 τ_i 出现的概率等于每个状态 s 出现的概率的累乘：

$$p_\theta(\tau_i) = p(s_{i,1})[p_\theta(a_{i,1}|s_{i,1})p(s_{i,2}|a_{i,1}, s_{i,1})][p_\theta(a_{i,2}|s_{i,2})p(s_{i,3}|a_{i,2}, s_{i,2})]... \tag{3.9}$$

中国知网 https://www.cnki.net

$$p_\theta(\tau_i) = p(s_{i,1}) \prod_{t=1}^{T} [p_\theta(a_{i,t}|s_{i,t}) p(s_{i,t+1}|s_{i,t}, a_{i,t})] \tag{3.10}$$

其中，状态转移概率 $p(s_{i,t+1}|s_{i,t}, a_{i,t})$ 与环境有关，与策略函数无关，因此与 θ 无关。而 $p_\theta(a_{i,t}|s_{i,t})$ 在状态 $s_{i,t}$ 选择动作 $a_{i,t}$ 的概率由 Pointer Networks 的模型参数 θ 决定：

$$\begin{aligned} \nabla_\theta log p_\theta(\tau_i) &= \nabla_\theta log[p(s_{i,1}) \prod_{t=1}^{T} [p_\theta(a_{i,t}|s_{i,t}) p(s_{i,t+1}|s_{i,t}, a_{i,t})] \\ &= \sum_{t=1}^{T} \nabla_\theta log p_\theta(a_{i,t}|s_{i,t}) \end{aligned} \tag{3.11}$$

通过当前模型参数 θ 下的 Pointer Networks 我们可以求 $p_\theta(a_{i,t}|s_{i,t})$ 的值，在计算得到 $R(\tau_i)$ 和 $\nabla_\theta log p_\theta(\tau_i)$ 后，我们便可以采集大量训练样本计算 $\bar{R}_\theta$ 的梯度：

$$\nabla_\theta \bar{R}_\theta \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} [\nabla_\theta log p_\theta(a_{i,t}|s_{i,t}) R(\tau_i)] \tag{3.12}$$

计算得到 $\bar{R}_\theta$ 的梯度后，便可更新模型参数 θ，再在新的 θ 下采集样本计算梯度再更新模型参数 θ，如此训练，直到得到合适的 θ 后停止。优化结束后的 Pointer Networks 便是我们所要求的策略函数。

3.3.2 Baseline 函数

模型的优化过程其实是在不断调整参数来提高好的行为出现的可能性，降低差的行为出现的可能性。而如果所有的行为的返回奖励都是正的，在理想情况下这种情况不会构成问题。如图3.7中的理想情况所示，假设在某一个状态 s 我们可以采取三个行为 a, b, c，对应的概率分别是 $0.3, 0.5, 0.2$，采取其中任意一个行为的返回奖励都是正的，但是返回的奖励大小不一，如箭头长度所示。行为 a, c 的返回奖励比较大，行为 c 的返回奖励比较小，经过参数的更新，行为 b 出现的几率降低，行为 a, c 出现的几率增加。

然而在实际采样过程中，假设在某一个状态 s 我们可以采取三个行为 a, b, c，对应的概率分别是 $0.3, 0.5, 0.2$，但是我们仅仅采样到了行为 b, c。由于行为 a 没有被采样收集，我们并不知道采取行为 a 所返回的奖励是多少，因此经过参数的更新，行为 a 出现的几率降低，行为 b, c 出现的几率增加。一个简单的方法避免该问题便是令返回的奖励值有正有负，不要都是正的。因此我们引入偏移量

中国知网 https://www.cnki.net

Baseline，用 b 表示，使总奖励值减去偏移量 b：

$$\nabla_\theta \bar{R}_\theta = \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}[(R(\tau_i)-b)\nabla_\theta log p_\theta(a_{i,t}|s_{i,t})] \tag{3.13}$$

若某一个采样轨迹 τ_i 特别好，即 $R(\tau_i) > b$，则提高其对应的几率，若 $R(\tau_i) < b$，则降低其对应的几率。通过引入偏移量 b，便可避免某一个行为由于没有被采样收集而导致几率降低。

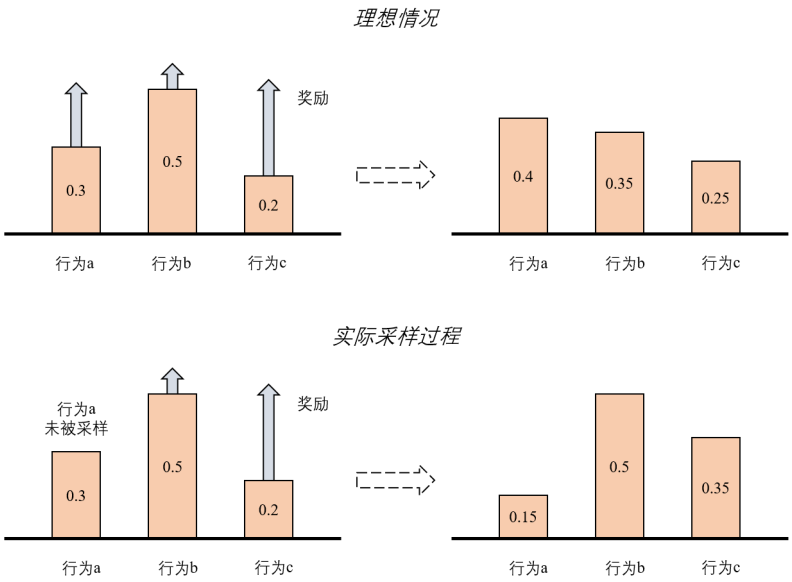


图 3.7 实际操作中遇到的问题

偏移量 b 的设计有很多选择，通常选择选择 $R(\tau)$ 的期望作为偏移量：

$$\begin{aligned} b &= E[R(\tau)] \\ &\approx \frac{1}{N}\sum_{i=1}^{N} R(\tau_i) \end{aligned} \tag{3.14}$$

引入 Baseline 之后，所有时刻的奖励均值均变为 0，因此产生返回奖励有正有负的行为，同时奖励值作为目标函数的权重，其绝对值得到一定的缩小，对算法的稳定性也有一定帮助。算法优化流程如3.1所示。

中国知网 https://www.cnki.net

算法 3.1 算法优化过程

训练数据集 S，迭代次数 T，一次迭代所选取的样本数 B
初始化 Pointer Networks 模型参数 θ
初始化偏移量 b
for $t = 1$ to T **do**
    从训练数据集 S 中选取一次迭代所需样本 $s_i, i \in \{1, ..., B\}$
    通过 $p_\theta(\cdot|\theta)$ 为每一个样本 $s_i, i \in \{1, ..., B\}$ 生成解方案 o_i
    计算 $\nabla_\theta \bar{R}_\theta = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} [(R(\tau_i) - b) \nabla_\theta log p_\theta(a_{i,t}|s_{i,t})]$
    更新模型参数 $\theta = Adam(\theta, \nabla_\theta \bar{R}_\theta)$
    更新偏移量 b
end for
返回 Pointer Networks 模型参数 θ

其中，$Adam$ 算法通过训练样本迭代地对神经网络中的参数进行更新，是一种可取代传统的梯度下降法的一阶优化算法 [27]。与传统的梯度下降法不同的是，在训练过程中，随机梯度下降法通过固定的学习率对网络中参数进行更新，而 $Adam$ 算法通过计算估计梯度的一阶矩和二阶矩来为神经网络中不同的权重提供不同的学习率，它的学习率是自适应的。$Adam$ 可以快速实现优质的结果，经验性结果表明 $Adam$ 算法在解决实际问题中拥有出色的表现，因此本文采用 $Adam$ 算法替代传统的随机梯度下降算法。

3.4 Bottom-Left-Fill 算法

在测试阶段，训练完毕的 Pointer Networks 将返回小矩形的放置顺序，本文根据构造类启发式 Bottom-Left-Fill 算法根据模型返回的放置顺序对小矩形进行放置。

Bottom-Left 算法在初始阶段将小矩形放置到矩形条的右上方，并使小矩形不断交替地向下向左移动，直至小矩形在当前所处位置不能继续移动，而该位置便是 Bottom-Left 算法为该小矩形安排的放置位置。而 Bottom-Left-Fill 算法在 Bottom-Left 算法的基础上做了进一步的优化，在向下向左搜索放置位置的过程的同时将先前放置小矩形产生的空白区域也考虑在内，Bottom-Left-Fill 算法流程如下3.2所示。

中国知网 https://www.cnki.net

算法 3.2 Bottom-Left-Fill 算法

初始化二维坐标系，矩形条处于第一象限且左下顶点处于原点 $(0,0)$
初始化小矩形的个数为 n，小矩形放置顺序为 $\{[w_1,h_1],[w_2,h_2],...,[w_n,h_n]\}$
初始化矩形条的宽为 W
初始化已放置小矩形列表 $recs=[[0,0,w_1,h_1]]$，（该列表由向量元素构成，每个元素第一第二维度表示小矩形左下顶点在矩形条中的横纵坐标，第三第四维度表示小矩形的宽和高）
初始化可放置底边列表 $bottom=[[0,0,W]]$，初始化 m 为可放置底边列表长度，（该列表由向量元素构成，每个元素第一第二维度表示该底边左顶点的横纵坐标，第三维度表示该底边的宽）
for $i=2$ to n **do**
    更新可放置底边列表 $bottom$ 和列表长度 m，并按照列表元素第二维对元素从小到大进行排列
    for $j=1$ to m **do**
        放置小矩形 $[w_i,h_i]$，使其左下顶点处于矩形条 $[0,bottom[j][2]]$ 位置
        $t=1$
        while $t\neq 0$ **do**
            $t=0$
            for $k=1$ to $i-1$ **do**
                通过重叠检测算法3.3检测小矩形 $[w_i,h_i]$ 与小矩形 $[recs[k][3],recs[k][4]]$ 是否重叠，并返回 ratio
                if $ratio>0$ **then**
                    则将小矩形 $[w_i,h_i]$ 向右平移到恰好不与小矩形 $[recs[k][3],recs[k][4]]$ 重叠为止
                    $t=1$
                    break
                end if
            end for
        end while
        if 移动后的小矩形 $[w_i,h_i]$ 未超出矩形条右边界 **then**
            break
        end if
    end for
    更新已放置小矩形列表 $recs$
end for

中国知网 https://www.cnki.net

算法 3.3 重叠检测算法

训练小矩形 $rec1 = [x_1, y_1, w_1, h_1], rec2 = [x_2, y_2, w_2, h_2]$，（其中 x 和 y 是小矩形左下顶点的横纵坐标值）
计算 $end_x = max(x_1 + w_1, x_2 + w_2), start_x = min(x_1, x_2)$
计算 $end_y = max(y_1 + h_1, y_2 + h_2), start_y = min(y_1, y_2)$
计算 $width = w_1 + w_2 - (end_x - start_x), height = h_1 + h_2 - (end_y - start_y)$
if $width \leq 0$ or $height \leq 0$ **then**
    $ratio = 0$
else
    $ratio = 1$
end if
返回 $ratio$

3.5 本章小结

本章首先对强化学习进行了概述，介绍了本文所用的基于策略的强化学习方法，并详细介绍了算法优化过程中的策略梯度算法以及梯度计算的数学推导过程。本章介绍了在实际操作中可能遇到的问题并引入 Baseline 算法进行优化。本章给出了算法优化过程的算法流程图和所用的 Adam 梯度更新算法的简单介绍。本章最后给出了本文装箱过程中所用的构造类启发式 Bottom-Left-Fill 算法，并展示了 BLF 的算法流程图。

中国知网 https://www.cnki.net

第四章　实验设计及结果分析

本章针对在第二章介绍的深度神经网络模型 Pointer Networks 和在第三章介绍的基于深度强化学习的 2D-SPP 算法进行相关的实验，并对实验结果进行对比分析。

4.1　实验设计

4.1.1　实验环境

本文所实现的基于深度强化学习的 2D-SPP 算法关于模型的训练和预测都在 GPU 上进行，算法的运行需要同时调动 CPU 和 GPU 的计算能力，实验对硬件的要求相对较高。

本文进行实验的软硬件环境如表4.1所示：

表 4.1 实验环境

OS 版本	Ubuntu 16.04 LTS
集成开发环境	JetBrains PyCharm 2019
深度学习框架	PyTorch 0.3
Python 版本	Python 3.6
CPU	Intel Core i7-9700K
GPU	Nvidia GEFORCE GTX 1080 Ti
RAM	USCORSAIR DDR4 3200 16GB(8G×2)

4.1.2　实验参数设置

在实验过程中，我们将一次迭代所选取的样本数设置为 128，LSTM 的隐藏单元设置为 128，词向量的维度设置为 128。模型采用 $Adam$ 优化算法来训练，学习率设置为 10^{-4}，设置学习率以指数衰减的方式进行更新，衰减系数为 0.96，衰减周期为 5000 次迭代，并设置最大梯度范数为 2 的梯度裁剪。Pointer Networks 中的所有参数通过均匀分布的随机初始化器从区间 $[-0.08, 0.08]$ 中生成。迭代周期设置为 20，当每个迭代周期执行完成后，将自动保存当前模型的参数，以便

中国知网　https://www.cnki.net

后期分析。

4.2 实验结果分析

4.2.1 算法的可行性与稳定性分析

首先我们分析算法的稳定性。以小矩形规模为 20 的 2D-SPP 为例，在训练过程中的每一次迭代，策略函数 Pointer Networks 分别对 128 个样本进行决策，计算总奖励值，并返回策略。图4.1所示是训练过程中一次迭代所有样本的总奖励的平均值随训练迭代步数变化趋势。考虑到本文所研究的 2D-SPP 的实际结果是矩形条的最终高度越小越好，因此为了更加直观地展示训练过程中的模型变化趋势，在图4.1中已将总奖励的平均值转化成矩形条高度的平均值。如图4.1所示，在训练初始阶段，Pointer Networks 给出的策略十分不理想，如图4.2所示，矩形条中的剩余空间浪费非常多，致使矩形条最终高度的平均值保持在 30 以上。随着训练过程的推进，Pointer Networks 通过不断学习优化策略，矩形条最终高度的平均值逐渐下降并收敛，趋于样本的最优解。

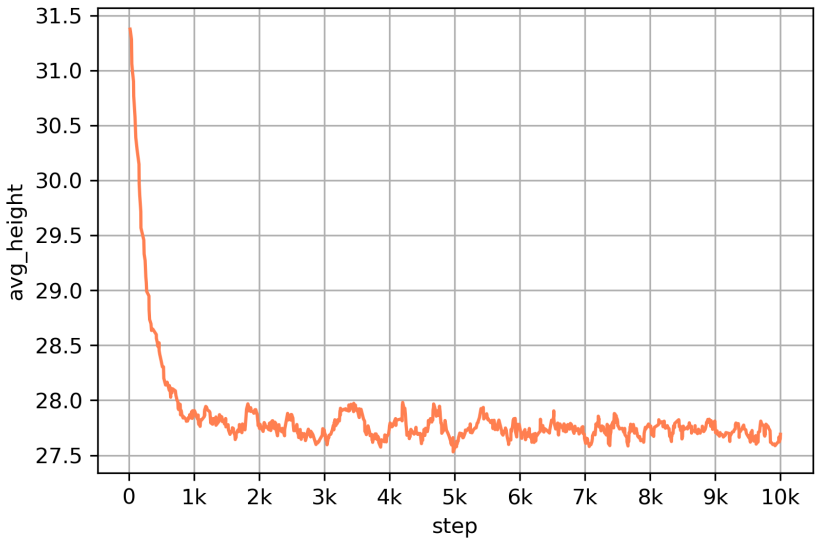


图 4.1 训练过程平均矩形条高度变化趋势

其次我们分析算法的可行性。以规模为 20 的 2D-SPP 为例，小矩形的尺寸分别为：$\{\langle 0 : (9.0, 8.0)]\rangle, \langle 1 : (6.0, 8.0)\rangle, \langle 2 : (7.0, 2.0)\rangle, \langle 3 : (7.0, 2.0)\rangle, \langle 4 : (5.0, 2.0)\rangle, \langle 5 : (9.0, 1.0)\rangle, \langle 6 : (8.0, 9.0)\rangle, \langle 7 : (4.0, 8.0)\rangle, \langle 8 : (7.0, 5.0)\rangle, \langle 9 : (3.0, 8.0)\rangle, \langle 10 : (3.0, 4.0)\rangle, \langle 11 : (4.0, 8.0)\rangle, \langle 12 : (9.0, 5.0)\rangle, \langle 13 : (9.0, 1.0)\rangle, \langle 14 : (3.0, 5.0)\rangle, \langle 15 :$

中国知网 https://www.cnki.net

$(9.0, 2.0)\rangle, \langle 16 : (1.0, 8.0)\rangle, \langle 17 : (6.0, 5.0)\rangle, \langle 18 : (2.0, 6.0)\rangle, \langle 19 : (7.0, 4.0)\rangle\}$。图4.2展示的是训练初始时 Pointer Networks 对该问题返回的比较坏的决策。图4.3展示的是 Pointer Networks 通过学习不断优化后返回的决策，其中图中所示的“()/()”，左边数字表示该小矩形在放置序列中的序号，右边数字表示该小矩形原本序号。Pointer Networks 通过学习不断优化后返回放置策略：$\{6, 0, 1, 12, 11, 7, 9, 8, 17, 19, 16, 18, 14, 10, 15, 2, 3, 4, 13, 5\}$，Bottom-Left-Fill 算法按照该策略对矩形条进行填充便可得到图4.3所示结果。可见，学习之后的模型返回的策略远好于先前的策略，并趋于最优解，说明了算法的可行性。

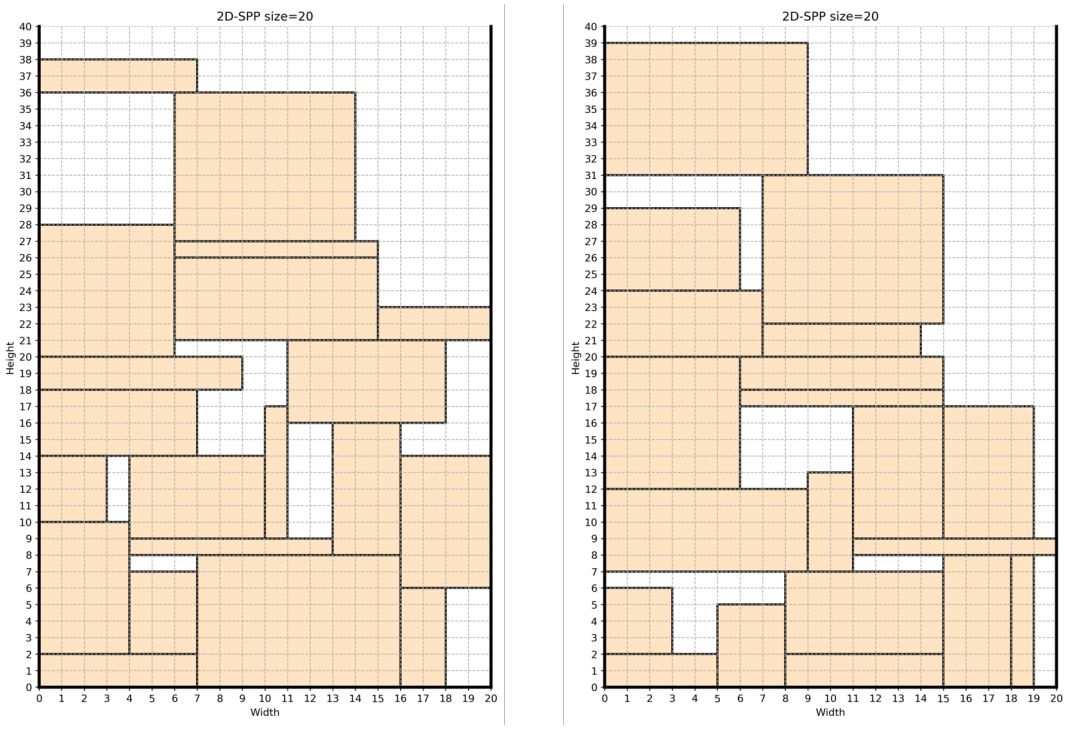


图 4.2 训练初始时 Pointer Networks 返回的比较坏的决策

最后我们分析算法在决策过程中是否准确合理，图4.5右图为 Pointer Networks 在返回图4.3所示策略过程中的输出概率分布的可视化。图4.5右图中上侧展示的分别是所有小矩形的尺寸，左侧展示的分布是策略函数 Pointer Networks 所选择的行为（即小矩形），每一行展示的是 Pointer Networks 每次选择行为时的概率分布，当行为对应的概率越小时，方格颜色显示越接近黑色，概率越大时，方格颜色显示越接近白色，如图右侧所示。我们对 Pointer Networks 在时刻 $t = 10$ 采取的行为进行分析。图4.4展示的是 Pointer Networks 在选择行为前后的矩形条布局。其中 Pointer Networks 在时刻 $t = 10$ 可采取的行为如图4.5左所

中国知网 https://www.cnki.net

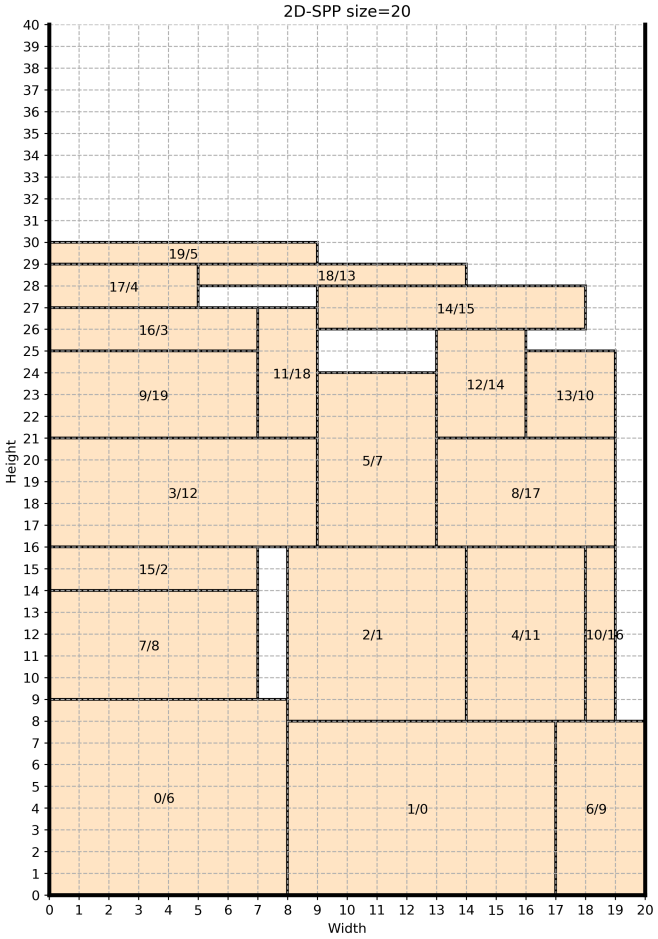


图 4.3 Pointer Networks 通过学习不断优化决策

示，$\{2,3,4,5,10,13,14,15,16,18\}$ 为还没放置的小矩形，图4.4左展示的是在时刻 $t=10$ 的矩形条布局。据图4.5右所示，小矩形 $\langle 16:(1.0,8.0)\rangle, \langle 18:(2.0,6.0)\rangle$ 对应的概率明显高于其它小矩形，而这两个小矩形也是所有剩余小矩形中适合放入先前放置过程中产生的剩余空间且不会造成矩形条高度的增加。而在这两个小矩形中，小矩形 $\langle 16:(1.0,8.0)\rangle$ 的概率明显大于小矩形 $\langle 18:(2.0,6.0)\rangle$，这一点也十分符合逻辑，因为填充小矩形 $\langle 16:(1.0,8.0)\rangle$ 后产生的剩余空间更加的规则，便于后续的填充，这体现了算法在进行决策时的合理性。

4.2.2 与启发式搜索算法的比较

本文最后使用 2D-SPP 的经典数据集对基于深度强化学习的 2D-SPP 求解算法进行测试，并与常用的启发式搜索算法进行比较。

中国知网 https://www.cnki.net

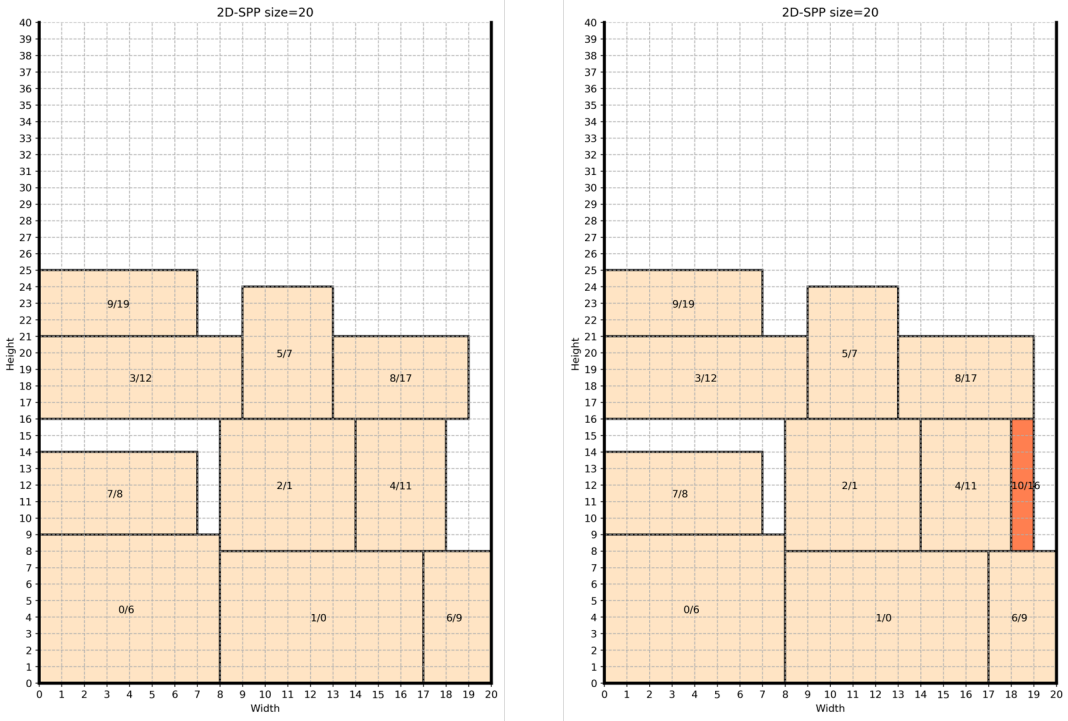


图 4.4 时刻 $t = 10$ Pointer Networks 选择行为前后的矩形条布局

4.2.2.1 数据集 C

本文选择数据集 C 对本文算法效果进行验证，该数据集由 Hopper (2000)[21]提出，并命名为 C。数据集 C 中共包含 21 个问题的数据，问题的规模从 17 到 197 不等，根据数据集中问题数据规模的不同，可将数据集分为 7 类，分别为 C_1~C_7，其中每类分别有 3 个问题的数据，以 C_4 为例，具体数据详见附录表a。数据集 C 中每个问题的小矩形长宽通过随机生成且最大长宽比为 7，且每个问题的最优高度已知，如表4.2所示：

表 4.2 数据集 C 问题类型

问题类型	问题规模	最优高度	矩形条宽度
C_1	16/17	20	20
C_2	25	15	40
C_3	28/29	30	60
C_4	49	60	60
C_5	72/73	90	60
C_6	97	120	80
C_7	196/197	240	160

中国知网 https://www.cnki.net

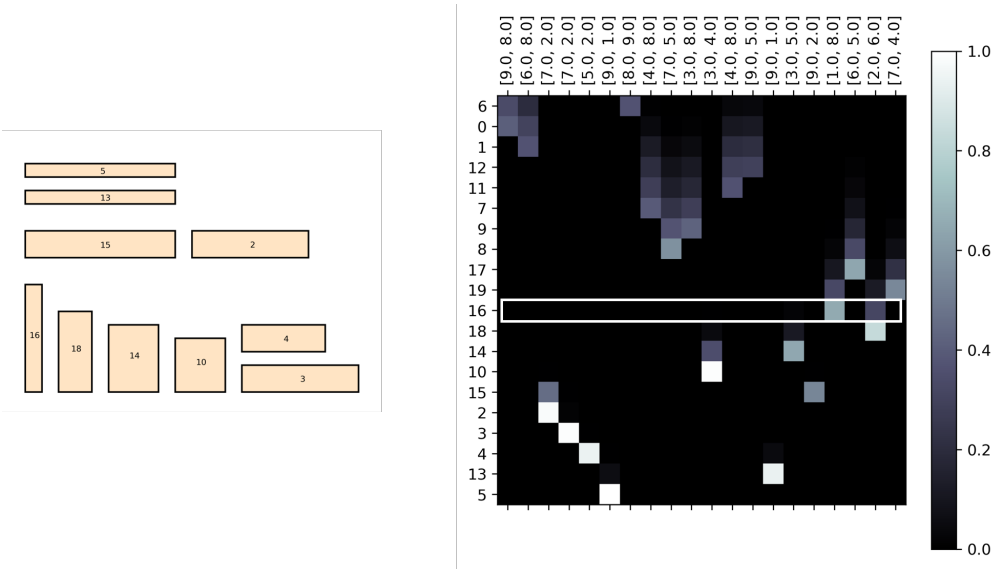


图 4.5 时刻 $t = 10$ 可采取的行为和 Pointer Networks 输出概率分布可视化

4.2.2.2 启发式搜索算法

随机搜索算法 RS(Random Search)：随机搜索算法是最简易的优化搜索算法，正如其名所述，进行随机的搜索大量可行解，并对每个可行解计算其相应的代价，在本文即指矩形条所使用的高度，并选择矩形条所使用高度最小的可行解作为随机搜索算法的最优解进行输出。

模拟退火算法 SA(Simulated Annealing)：模拟退火来自冶金学的专有名词退火。从一个问题的原始解开始，用一个变量代表温度，这一温度开始时非常高，而后逐步减低。在每一次迭代期间，算法会随机选中解中的某个数字，使其发生细微变化，而后计算该解的代价。在计算出该解的代价后，如果新的成本更低，则新的解就会变成当前解，与爬山法类似；如果新的成本更高，则以概率 p 被接受。这一概率会随着温度 T 的降低而降低。即算法开始时，可以接受表现较差的解，随着退火过程中温度的不断下降，算法越来越不可以接受较差的解，直到最后，它只会接受更优的解。

遗传算法 GA(Genetic Algorithm)：遗传算法是一种通过模拟自然进化过程搜索最优解的启发式搜索算法，它是模拟达尔文生物进化论的自然选择和遗传学机理的生物进化过程的计算模型。遗传算法将种群中所有个体作为对象，并利用随机化技术指导对一个被编码的参数空间进行高效搜索。选择、交叉和变异是构成遗传算法的三种最主要的操作；遗传操作设计、适应度函数设计、初始群体设

中国知网 https://www.cnki.net

定、控制参数设定、参数编码则是组成遗传算法核心内容的五个要素。

4.2.2.3 实验结果

随机搜索算法随机搜索 50000 个可行解并选择其中的最优解。遗传算法的种群大小设置为 50，进化代数设置为 1000。模拟退火算法的终止条件为连续若干个新解都没有被接受。由于硬件设施的限制，本文没有对 C_7 的三个问题进行测试，启发式搜索算法求解 C_7 问题规模的估计时间在 10h~60h 之间不等，本文深度强化学习算法在一次迭代样本数为 128，迭代步数为 1000 的参数环境下，估计训练时间为 270h，且在训练过程中容易出现因为内存不足而导致程序终止的情况。因此本文算法仅在 C_7 问题上进行了简单的训练（一次迭代样本数为 128，迭代步数为 10），并对训练后的模型的线上运行速度进行了测试。启发式搜索算法与本文深度强化学习算法 (Deep Reinforcement Learning, DRL) 的实验结果如表4.3所示，表4.3中给出的是算法结果与最优解的相对距离：(算法结果-最优解)/最优解。可见，本文算法在求解不同规模的 2D-SPP 上的实验结果都稳定在 5%~10% 之间，趋于最优解。图4.6为本文算法在 C_4 问题上的实验结果。

表 4.3 算法结果与最优解的相对距离（%）

	C_1	C_2	C_3	C_4	C_5	C_6	C_7
RS+BLF	5	13.3	16.7	13.3	11.1	16.7	-
SA+BLF	5	0	10	10	7.8	8.3	-
GA+BLF	10	13.3	13.3	10	10	9.2	-
DRL+BLF	10	6.7	10	6.7	6.7	5.8	-

同时，本文对各个算法的线上运行速度进行了测试，如表4.4所示，其中秒 (s) 与毫秒 (ms) 之间的单位换算为：$1s = 1000ms$。如表4.4所示，在小规模的问题上，本文算法的运行速度是毫秒级的，启发式搜索算法在小规模问题上的运行速度同样不错。而在处理大规模的问题时，本文算法的线上运行速度显著优于启发式搜索算法，即使在最大规模的 C_7 问题上，本文算法也可以在 10s 之内返回装箱策略。因此，本文算法在求解小规模的 2D-SPP 时，相较于启发式搜索算法优势不明显，而在求解大规模的 2D-SPP 时，在装箱策略普遍优于或近似于启发式搜索算法的同时，本文算法线上计算复杂度远远低于启发式搜索算法，运行速度极快。同样，深度强化学习算法的缺陷也很明显，对硬件设施要求很高，线下训练阶段需要大量时间完成。

中国知网 https://www.cnki.net

表 4.4 算法线上平均运行时间

	C_1	C_2	C_3	C_4	C_5	C_6	C_7
RS	0.77s	0.78s	0.81s	2.34s	7.13s	16.48s	118.51s
SA	0.94s	3.68s	8.43s	59.45s	237.61s	890.19s	-
GA	1min	2min	3min	13min	36min	86min	-
DRL	35ms	42ms	47ms	0.15s	0.42s	0.95s	7.13s

4.3 本章小结

本章首先介绍了本文的实验设计，对本文算法的可行性与稳定性进行分析并对实验结果进行展示。最后在 2D-SPP 的经典数据集 C 上对本文算法进行测试，并与启发式搜索算法在实验结果和算法线上运行速度等方面进行了比较，并总结了算法的优缺点。

中国知网 https://www.cnki.net

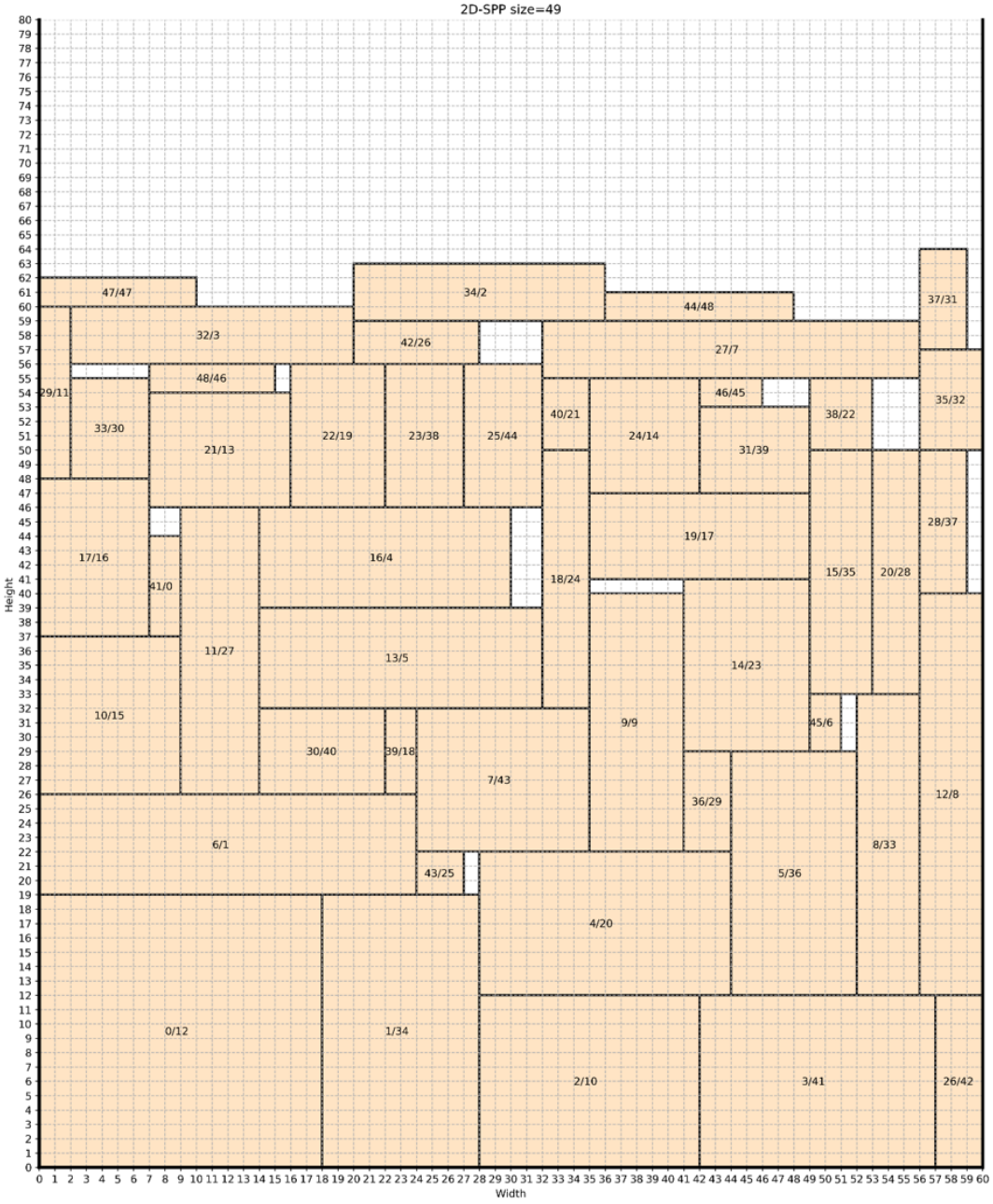


图 4.6 本文算法在 C_4 上的实验结果

中国知网 https://www.cnki.net

中国知网 https://www.cnki.net

总结与展望

2D-SPP 作为装箱问题的一种，在很多领域都有着广泛且重要的应用。本文主要探索了基于深度强化学习的 2D-SPP 求解算法。本文首先在第二章介绍了深度强化学习算法的深度学习部分。通过循环神经网络、Seq2Seq 模型和注意力机制的描述，对本文所用的深度神经网络模型 Pointer Networks 及其特点进行了详细的介绍，并介绍了在模型测试阶段使用的 Beam Search 算法。随后在第三章本文介绍了深度强化学习算法的强化学习部分。首先对强化学习进行了概述，并介绍了本文所用的基于策略的强化学习方法，包括策略函数和奖励函数的设计等。同时介绍了算法优化过程中的策略梯度算法以及梯度计算的数学推导过程，并介绍了实际操作中可能遇到的问题以及通过 Baseline 函数进行优化，最后给出了算法优化过程的算法流程图。本文最后在第四章通过实验对本文算法的性能进行评估，对算法的可行性与稳定性进行了分析。并通过 2D-SPP 的经典数据集 C 对算法进行测试，本文算法在求解不同规模的 2D-SPP 上的实验结果都稳定在 5%~10% 之间，趋于最优解。而在小规模的问题上，本文算法的线上运行速度是毫秒级的，即使在大规模问题上，本文算法也可以在数秒内返回装箱策略，在线上运行速度上，本文算法显著优于启发式搜索算法。本文通过大量实验展示了深度强化学习算法用于求解 2D-SPP 的可行性与优势，为今后求解装箱问题提供了一种全新的解决思路。

在本文工作的基础上，后续探索方向主要有：1. 本文算法在 2D-SPP 上取得不错的结果，后续可将基于深度强化学习的求解算法应用到其他装箱问题上，并与启发式算法进行各方面的比较；2. 在强化学习方法和策略函数上进行更深入的研究，探索是否可以进一步提升模型效果。

中国知网 https://www.cnki.net

中国知网 https://www.cnki.net

参考文献

[1] Alvarez-Valdés, R., Parreño, F., Tamarit, J.M.: A grasp algorithm for constrained two-dimensional non-guillotine cutting problems. Journal of the Operational Research Society **56**(4), 414–425 (2005)

[2] Alvarez-Valdés, R., Parreño, F., Tamarit, J.M.: Reactive grasp for the strip-packing problem. Computers & Operations Research **35**(4), 1065–1083 (2008)

[3] Bahdanau, D., Cho, K., Bengio, Y.: Neural machine translation by jointly learning to align and translate. arXiv preprint arXiv:1409.0473 (2014)

[4] Baker, B.S., Coffman Jr, E.G., Rivest, R.L.: Orthogonal packings in two dimensions. SIAM Journal on computing **9**(4), 846–855 (1980)

[5] Beasley, J.: An exact two-dimensional non-guillotine cutting tree search procedure. Operations Research **33**(1), 49–64 (1985)

[6] Bello, I., Pham, H., Le, Q.V., Norouzi, M., Bengio, S.: Neural combinatorial optimization with reinforcement learning. arXiv preprint arXiv:1611.09940 (2016)

[7] Berkey, J.O., Wang, P.Y.: Two-dimensional finite bin-packing algorithms. Journal of the operational research society **38**(5), 423–429 (1987)

[8] Bortfeldt, A.: A genetic algorithm for the two-dimensional strip packing problem with rectangular pieces. European Journal of Operational Research **172**(3), 814–837 (2006)

[9] Buchwald, T., Scheithauer, G.: Upper bounds for heuristic approaches to the strip packing problem. International Transactions in Operational Research **23**(1-2), 93–119 (2016)

[10] Burke, E.K., Hyde, M., Kendall, G., Woodward, J.: A genetic programming hyper-heuristic approach for evolving 2-d strip packing heuristics. IEEE Transactions on Evolutionary Computation **14**(6), 942–958 (2010)

[11] Burke, E.K., Hyde, M.R., Kendall, G.: A squeaky wheel optimisation methodology for two-dimensional strip packing. Computers & Operations Research **38**(7), 1035–1044 (2011)

[12] Burke, E.K., Kendall, G., Whitwell, G.: A new placement heuristic for the orthogonal stock-cutting problem. Operations Research **52**(4), 655–671 (2004)

[13] Chazelle, B.: The bottomn-left bin-packing heuristic: An efficient implementation. IEEE Transactions on Computers (8), 697–707 (1983)

[14] Chen, J., Zhu, W., Peng, Z.: A heuristic algorithm for the strip packing problem. Journal of Heuristics **18**(4), 677–697 (2012)

中国知网 https://www.cnki.net

[15] Christofides, N., Hadjiconstantinou, E.: An exact algorithm for orthogonal 2-d cutting problems using guillotine cuts. European Journal of Operational Research **83**(1), 21–38 (1995)

[16] Coffman Jr, E.G., Garey, M.R., Johnson, D.S., Tarjan, R.E.: Performance bounds for level-oriented two-dimensional packing algorithms. SIAM Journal on Computing **9**(4), 808–826 (1980)

[17] Cui, Y., Yang, Y., Cheng, X., Song, P.: A recursive branch-and-bound algorithm for the rectangular guillotine strip packing problem. Computers & Operations Research **35**(4), 1281–1291 (2008)

[18] Hadjiconstantinou, E., Iori, M.: A hybrid genetic algorithm for the two-dimensional single large object placement problem. European Journal of Operational Research **183**(3), 1150–1166 (2007)

[19] Hochbaum, D.S., Maass, W.: Approximation schemes for covering and packing problems in image processing and vlsi. Journal of the ACM (JACM) **32**(1), 130–136 (1985)

[20] Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural computation **9**(8), 1735–1780 (1997)

[21] Hopper, E.: Two-dimensional packing utilising evolutionary algorithms and other meta-heuristic methods. Ph.D. thesis, University of Wales. Cardiff (2000)

[22] Hopper, E., Turton, B.: A genetic algorithm for a 2d industrial packing problem. Computers & Industrial Engineering **37**(1-2), 375–378 (1999)

[23] Hopper, E., Turton, B.C.: An empirical investigation of meta-heuristic and heuristic algorithms for a 2d packing problem. European Journal of Operational Research **128**(1), 34–57 (2001)

[24] Hu, H., Zhang, X., Yan, X., Wang, L., Xu, Y.: Solving a new 3d bin packing problem with deep reinforcement learning method. arXiv preprint arXiv:1708.05930 (2017)

[25] Imahori, S., Yagiura, M.: The best-fit heuristic for the rectangular strip packing problem: An efficient implementation and the worst-case approximation ratio. Computers & Operations Research **37**(2), 325–333 (2010)

[26] Jakobs, S.: On genetic algorithms for the packing of polygons. European journal of operational research **88**(1), 165–181 (1996)

[27] Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980 (2014)

[28] Leung, S.C., Zhang, D.: A fast layer-based heuristic for non-guillotine strip packing. Expert Systems with Applications **38**(10), 13032–13042 (2011)

中国知网 https://www.cnki.net

[29] Leung, S.C., Zhang, D., Sim, K.M.: A two-stage intelligent search algorithm for the two-dimensional strip packing problem. European Journal of Operational Research **215**(1), 57–69 (2011)

[30] Liu, D., Teng, H.: An improved bl-algorithm for genetic algorithm of the orthogonal packing of rectangles. European Journal of Operational Research **112**(2), 413–420 (1999)

[31] Lodi, A., Martello, S., Vigo, D.: Heuristic and metaheuristic approaches for a class of two-dimensional bin packing problems. INFORMS Journal on Computing **11**(4), 345–357 (1999)

[32] Martello, S., Monaci, M., Vigo, D.: An exact approach to the strip-packing problem. INFORMS Journal on Computing **15**(3), 310–319 (2003)

[33] Neveu, B., Trombettoni, G., Araya, I., Riff, M.C.: A strip packing solving method using an incremental move based on maximal holes. International Journal on Artificial Intelligence Tools **17**(05), 881–901 (2008)

[34] Ntene, N.: An algorithmic approach to the 2d oriented strip packing problem. Ph.D. thesis, Stellenbosch: Stellenbosch University (2007)

[35] Ortmann, F.G., Ntene, N., Van Vuuren, J.H.: New and improved level heuristics for the rectangular strip packing and variable-sized bin packing problems. European Journal of Operational Research **203**(2), 306–315 (2010)

[36] Özcan, E., Kai, Z., Drake, J.H.: Bidirectional best-fit heuristic considering compound placement for two dimensional orthogonal rectangular strip packing. Expert Systems with Applications **40**(10), 4035–4043 (2013)

[37] Pisinger, D.: Denser packings obtained in o (n log log n) time. INFORMS Journal on Computing **19**(3), 395–405 (2007)

[38] Salto, C., Alba, E., Molina, J.M., Leguizamón, G.: Greedy seeding procedure for gas solving a strip packing problem. Inteligencia Artificial. Revista Iberoamericana de Inteligencia Artificial **12**(40), 73–85 (2008)

[39] Scheithauer, G.: Equivalence and dominance for problems of optimal packing of rectangles. Citeseer (1995)

[40] Sutskever, I., Vinyals, O., Le, Q.: Sequence to sequence learning with neural networks. Advances in NIPS (2014)

[41] Thomas, J., Chaudhari, N.S.: A new metaheuristic genetic-based placement algorithm for 2d strip packing. Journal of Industrial Engineering International **10**(1), 47 (2014)

[42] Verstichel, J., De Causmaecker, P., Berghe, G.V.: An improved best-fit heuristic for the orthogonal strip packing problem. International Transactions in Operational

中国知网 https://www.cnki.net

Research **20**(5), 711–730 (2013)

[43] Vinyals, O., Fortunato, M., Jaitly, N.: Pointer networks. In: Advances in Neural Information Processing Systems, pp. 2692–2700 (2015)

[44] Wei, L., Oon, W.C., Zhu, W., Lim, A.: A skyline heuristic for the 2d rectangular packing and strip packing problems. European Journal of Operational Research **215**(2), 337–346 (2011)

[45] Wei, L., Qin, H., Cheang, B., Xu, X.: An efficient intelligent search algorithm for the two-dimensional rectangular strip packing problem. International Transactions in Operational Research **23**(1-2), 65–92 (2016)

[46] Wei, L., Tian, T., Zhu, W., Lim, A.: A block-based layer building approach for the 2d guillotine strip packing problem. European Journal of Operational Research **239**(1), 58–69 (2014)

[47] Wei, L., Zhang, D., Chen, Q.: A least wasted first heuristic algorithm for the rectangular packing problem. Computers & Operations Research **36**(5), 1608–1614 (2009)

[48] Yang, S., Han, S., Ye, W.: A simple randomized algorithm for two-dimensional strip packing. Computers & Operations Research **40**(1), 1–8 (2013)

[49] Zhang, D., Kang, Y., Deng, A.: A new heuristic recursive algorithm for the strip rectangular packing problem. Computers & Operations Research **33**(8), 2209–2217 (2006)

[50] Zhang, D., Shi, L., Leung, S.C., Wu, T.: A priority heuristic for the guillotine rectangular packing problem. Information Processing Letters **116**(1), 15–21 (2016)

[51] Zhang, D., Wei, L., Leung, S.C., Chen, Q.: A binary search heuristic algorithm based on randomized local search for the rectangular strip-packing problem. INFORMS Journal on Computing **25**(2), 332–345 (2013)

[52] Zhang, D.F., Sheng-Da, C., Yan-Juan, L.: An improved heuristic recursive strategy based on genetic algorithm for the strip rectangular packing problem. Acta Automatica Sinica **33**(9), 911–916 (2007)

中国知网 https://www.cnki.net

附录

数据集 C 中问题类型 C_4 的具体数据如下表a所示：

表 a 数据集 C 问题类型 C_4

P1				P2				P3			
w	h	w	h	w	h	w	h	w	h	w	h
2	7	3	3	10	14	2	4	10	4	12	7
24	7	8	3	3	13	2	7	12	4	9	4
16	4	5	20	28	5	3	4	13	5	4	4
18	4	3	17	5	8	5	30	3	5	9	9
16	7	3	7	14	9	5	3	7	22	2	5
18	7	5	7	12	14	10	26	6	22	20	5
2	4	3	7	13	10	6	5	9	23	9	5
24	4	4	7	3	17	4	9	10	19	4	5
4	28	4	21	1	5	1	4	3	15	4	2
6	18	10	19	4	1	9	2	5	13	12	2
14	12	4	17	18	4	4	17	2	10	3	15
2	12	8	17	1	1	5	2	2	10	21	11
18	19	3	10	2	6	4	4	13	18	11	3
9	8	5	10	4	14	6	2	3	18	3	3
7	8	7	6	3	18	4	10	2	3	11	23
9	11	8	6	4	14	2	4	2	3	11	23
7	11	15	12	8	17	3	12	5	2	11	8
14	6	3	12	11	5	6	5	4	2	3	8
2	6	11	10	9	12	3	9	3	4	21	4
6	10	5	10	4	7	7	18	9	4	14	4
16	10	4	2	25	8	6	6	7	1	3	13
3	5	8	2	7	5	18	7	6	1	35	13
4	5	10	2	24	9	13	9	2	4	11	5
8	12	12	2	9	14	25	7	20	4	11	5
3	18			12	19			4	7		

中国知网 https://www.cnki.net

中国知网 https://www.cnki.net

致谢

三年的时间看似很长，却如白驹过隙，在感叹时间飞逝的同时也感恩这段时光对我的锤炼。

感谢我的导师，吕长虹老师。吕老师不仅在科研上为我指点迷津，在生活中同样给予了我许许多多的教导和帮助。感谢老师在毕业论文上对我的指导，当我遇到困难时，老师为我解惑，给予我很大的帮助。老师您对数学的热爱和对学术研究的执着和严谨值得我一生学习。

感谢堵丁柱教授，给予我机会进入德克萨斯大学达拉斯分校计算机系访问学习。谢谢堵老师在科研方面对我贯彻始终的耐心指导，在生活和未来规划上给予我宝贵的建议。

感谢我的父母对我求学之路的支持，是你们的支持给予了我前进的动力。

感谢研究生期间遇到的各种朋友们，你们的陪伴让我这三年时光变得更加多彩。

在今后的求学之路上，我将带着大家对我的信任与支持不断前行，争取早日学有所成，为祖国健康工作 50 年。

中国知网 https://www.cnki.net